CellScope: A Tool for Capturing Cell-Level Metadata of Digital Objects in JupyterLab

Guillermo Camacho Hortelano
guillermo.camacho.hortelano@student.uva.nl

January 22, 2026, 48 pages

Academic supervisor: dr. Zhiming Zhao, z.zhao@uva.nl
Daily supervisor: Koen Greuell, koen.greuell@lifewatch.eu
Research group: MultiScale Networked Systems,
https://ivi.uva.nl/research/multiscale-networked-systems-mns.html

Universiteit van Amsterdam
Faculteit der Natuurwetenschappen, Wiskunde en Informatica
Master Software Engineering
http://www.software-engineering-amsterdam.nl

Abstract

Jupyter notebooks are widely used for exploratory and production research, yet results are often difficult to understand, reproduce, and reuse because key context remains implicit: which cells define or depend on a value, which artifacts are produced or consumed, and under what conditions outputs were generated. This thesis presents *CellScope*, a JupyterLab-first tool that captures richer cell-level metadata about notebook digital objects and packages it as a portable, web-native research object. CellScope performs static-first analysis to extract per-cell definitions and uses, file reads and writes, and inferred producer→consumer relationships, and exports an RO-Crate (JSON-LD) enriched with provenance links (PROV) and lightweight per-object profiles. The exported crate includes the notebook, cell snapshots, materialised artifacts, and an offline graph viewer, while an optional triple-store index provides fast discovery queries without becoming the source of truth.

We validate CellScope along three objectives: capture coverage, navigation efficiency, and interactive performance. Coverage evaluation against gold templates shows perfect results on controlled Python notebooks and highlights clear boundaries for polyglot and dynamic patterns (notably R parsing and variable-driven file paths). A within-subjects user study demonstrates that CellScope reduces time-to-answer for provenance and dependency questions compared to baseline notebook navigation, with large and consistent improvements after iterative usability refinements. Microbenchmarks show that analysis, export, and optional indexing remain within interactive bounds at the tested scales, and that indexed queries achieve low-latency responses. Overall, the results indicate that storage-backed, cell-level metadata can materially improve notebook understandability and support reuse-oriented packaging, while motivating future work on stronger polyglot support, persistent metadata capture, and assistive metadata suggestions.

Contents

Chapter 1

Introduction

In modern virtual labs, much of the day-to-day work lives in shared JupyterLab notebooks. A developer wants to clean up a large notebook by deleting a variable they think is obsolete, but first they have to scroll through dozens of cells to check whether some distant plot or export still depends on it. Another day, the same developer is asked to change a configuration file that controls how an analysis pipeline runs, and they need to understand which tables, models, and figures will be affected before they touch it. A collaborator later opens the notebook to reuse a single figure for a report, but cannot easily see which inputs, parameters, or intermediate steps gave rise to that result. In all of these scenarios, it becomes easy to make mistakes or misinterpret results as notebooks and pipelines grow: important contextual information about digital objects is missing, fragmented, or buried in code. Studies confirm this is common; many public notebooks are not directly runnable and need manual fixes or institutional knowledge [1–4]. Reproducibility and reuse get harder as the notebook grows.

1.1 Motivation

The day-to-day issues are simple and persistent:

- **Context disappears.** Figures and tables lose their origin. It is hard to trace a result back to the code, inputs, and parameters that produced it [5–7].
- **Dependencies are implicit.** Execution order diverges from cell order; variables live in hidden state; the same concept gets different names over time [1].
- **Sharing is brittle.** Copying a few cells rarely carries the *explanations* others need to run and adapt them.
- **FAIR is "somewhere else".** Guidance exists (e.g., Ten Simple Rules; FAIR principles), but tools are often outside Jupyter and metadata is handwritten [8, 9].

We want something small but concrete: make the key metadata and contextual information about these digital objects, including how they are produced and used across cells, *visible* and *portable* without asking authors to manually annotate every step.

1.2 Problem Statement

Even small notebooks produce many digital objects: code cells, variables, derived tables, figures, configuration files. In practice, these digital objects are what get shared, archived, or cited. However, most notebook tooling treats them as by-products of execution rather than first-class entities with metadata. Information about what each object represents, how it was created, and how it is connected to others remains implicit in code and execution state. Without a lightweight way to capture and package such contextual metadata:

- reproducibility is fragile (out-of-order execution and undeclared dependencies are hidden behind missing context),
- reuse is slow (reusers must read and re-execute code to reconstruct which digital objects depend on which inputs),
- and results management is manual (no machine-actionable metadata about digital objects and their provenance).

At a high level, NaaVRE is the surrounding environment for this work: it extends Jupyter with services aimed at turning notebook steps into reusable components and composing pipelines [4, 10]. This is valuable for making work run elsewhere. However, it does not maintain a notebook-wide record of the digital objects involved in an analysis and the contextual metadata that explains how they were produced, nor does it bundle that metadata into a portable, standards-based package that others can inspect and reuse [5, 11, 12]. CellScope addresses this gap by capturing cell-level metadata about notebook digital objects and packaging it for human inspection and machine-actionable reuse, while remaining compatible with NaaVRE as a downstream environment.

1.3 Research Questions

We scope the work to **Jupyter** notebooks and the digital objects they produce. NaaVRE is treated as a use case for integration, not as the primary scope.

> **Main RQ:** How can richer cell-level metadata and contextual information about digital objects in Jupyter notebooks improve the understandability, reproducibility, and reuse of computational results?

We explore three sub-questions:

- **RQ1.** Which kinds of cell-level metadata and contextual information about digital objects do notebook authors and reusers need in order to understand and reliably reproduce results? [13, 14]
- **RQ2.** How can this metadata be structured and packaged into a portable, web-native representation that tools can index and humans can inspect? [5, 11, 12]
- **RQ3.** What interface within JupyterLab best helps users *see* and *navigate* this metadata about digital objects directly from the notebook (e.g., interactive graphs and lightweight metadata search)? [15–18]

Why not just reuse the NaaVRE analyser? We align with it conceptually (analysis of notebooks without intrusive runtime instrumentation) but pursue a different deliverable: a portable, FAIR-oriented layer of cell-level metadata and contextual information about digital objects, plus a human-facing graph and list view embedded in JupyterLab. The implementation is designed to work both inside and outside NaaVRE so that the same packaged metadata can be reused in other environments.

1.4 Contributions

This thesis delivers:

1. **Cell-level metadata model** for digital objects in Python and R notebooks, covering code cells, variables, and derived artifacts, together with the contextual information needed to interpret them. *(RQ1)*
2. **Storage and indexing (confirm-first).** Authors review and confirm captured metadata before it becomes part of a persistent, portable research object, and a lightweight index supports cross-notebook and cross-project findability. *(RQ1→RQ2)*
3. **Portable packaging** of notebooks, digital objects, and their contextual explanations using a standards-based research object with explicit provenance relations between inputs, computational steps, and results [5, 11]. *(RQ2)*
4. **Interactive overview** in JupyterLab where a combined graph and list/filter view expose digital objects and their metadata, helping users gain an overview and quickly locate the context they need. *(RQ3)*

The prototype runs stand-alone and can be integrated into NaaVRE's notebook flow later. It *complements* the existing NaaVRE containerizer/analysers by adding portable, metadata-rich provenance packaging and graph-first exploration rather than replacing their containerization pipeline [10, 19].

1.5 Outline

Chapter 2 reviews the current state of metadata capture and storage, as well as existing solutions and their gaps. Chapter 3 explains the approach, requirements analysis, and a conceptual overview of the platform.

Chapter 4 details the technical implementation. Chapter 5 evaluates the results against relevant metrics. Chapter 6 discusses the achievement of objectives, reflects on the use cases, and summarises lessons learned from development and evaluation. Chapter 7 concludes the thesis by summarising the project and answering the research questions. Chapter 8 outlines limitations and directions for future work, informed by the lessons learned and observed gaps.

Chapter 2

State of the Art

Building on the motivation and research questions from Chapter 1, this chapter positions CellScope within the broader landscape of metadata, provenance, and notebook tooling. We first review background on packaging specifications, virtual research environments, and digital objects in Jupyter-based workflows (Section 2.1). We then survey and classify existing tools and platforms (Section 2.2 and Section 2.3), before analysing how they relate to our research questions and where a gap remains around cell-level metadata for digital objects (Section 2.4).

2.1 Background

We keep two threads in view. First, evidence: notebooks are popular but fragile in practice. Large-scale studies report common quality issues; hidden state, non-linear execution, missing dependencies, and frequent need for manual fixes [1–3]. Second, guidance: reproducibility frameworks emphasise environment capture, provenance, automation, and sharing [8, 9]. The gap is not a lack of principles; it is the distance between those principles and everyday notebook work.

Packaging and provenance. RO-Crate is a community specification for packaging data, code, and metadata as JSON-LD next to the actual files [11, 12]. It is web-native, aligns with schema.org, and can incorporate PROV for provenance [5] and DCAT for cataloging [20, 21]. Profiles for workflows and workflow runs allow us to describe inputs, outputs, parameters, and execution traces [22].

Where notebooks meet VREs. WorkflowHub and Galaxy show how to register and run workflows; they are strong at tool/workflow granularity [17, 23]. FAIRDOM-SEEK focuses on data/model assets and sharing [18]. NaaVRE demonstrates a notebook-centric cloud VRE [10]. What is generally missing is *cell-level* metadata capture and an in-place way to inspect those relationships inside Jupyter.

NaaVRE (technical summary). NaaVRE is a notebook-centric VRE that turns notebook steps into reusable components via a *component containerizer*. Two analysers matter for this thesis. First, a **Code Analyser / I/O Detector** that parses cells to infer inputs/outputs and code structure at *containerization time* (e.g., detect file and variable I/O, derive simple types) [10, 19]. Second, a lightweight **Type Detector** that can query the running kernel for types when needed through the Jupyter messaging protocol [4, 19]. These are effective for composing pipelines, but they do not persist a *notebook-wide* record of cell→data relationships nor package that record as a portable research object. Our work complements NaaVRE by *packaging* the resulting relationships as RO-Crate with PROV so they travel with the notebook. Beyond analysers, NaaVRE includes services such as an *experiment manager* (tracking executions and parameters) and a *marketplace/registry* for publishing components and artifacts. CellScope aims to complement these rather than duplicate them: it captures *intra-notebook* relationships, packages them as a portable research object, and exposes a minimal index suitable for discovery across projects [10, 24].

Digital objects in a Jupyter-based VRE. For clarity we explicitly enumerate the digital-object types used throughout this thesis (see Table 2.1). We keep a pragmatic list that reflects everyday notebook work and VRE practice [4, 10–12]. We distinguish *as-authored* objects (inside the notebook) from *packaged* objects (in the RO-Crate).

Type	Short definition & examples
Notebook	The `.ipynb` document as authored (cells, metadata).
Cell (code/markdown)	Executable or narrative unit; may define functions, variables, figures.
Variable / symbol	Logical data produced/used across cells (e.g., `df_clean`, `model`).
Parameter / config	Tunables that influence results (e.g., file paths, hyperparameters).
File / artifact	Concrete outputs/inputs on disk (e.g., `.csv`, `.parquet`, figures).
Dataset (curated)	A collection of files with stable identity and description (externally cited).
Figure / table	Rendered visual/tabular result derived from data and code.
Environment / kernel	Runtime image/interpreter that executes the notebook.
Component (containerized step)	A cell or group of cells packaged to run as a reusable step in a VRE [10].
Workflow (.naavrewf)	A NaaVRE workflow definition file used to compose components/pipelines (serializable document for orchestration).
Metadata record / Crate	The RO-Crate JSON-LD + sidecar files that carry human- and machine-readable context [11, 12].

Table 2.1: Digital objects considered in this thesis (11 types).

Note on secrets. Secret/credential files (e.g., tokens, passwords) are *not* included as digital objects for security and sharing reasons; instead, crates reference placeholders and rights notes, with resolution delegated to platform mechanisms (see Chapter 4).

Metadata profiles per object type. We reuse RO-Crate and PROV where possible and keep profiles small and practical [5, 11, 21]. Concretely, we start from the entity types and properties recommended by the RO-Crate specification (which itself builds on schema.org), add PROV relations for provenance (`prov:wasGeneratedBy`, `prov:used`), and bring in DCAT terms where dataset/catalog semantics are needed. OntoDT and domain conventions (e.g., CF Standard Names) inform how we describe data-like objects so that they can later participate in type-aware search and validation [25–27]. Table 2.2 maps each object type to core fields and vocabularies; the concrete *acquisition paths* (auto-captured vs. user-confirmed; NaaVRE integration points) are detailed in Chapter 4.

In practice, the field selection followed a simple procedure. First, we identified the digital-object types that appear in everyday notebook work (Table 2.1). Second, we mapped each type to a small set of schema.org/RO-Crate entity types that already have community support. Third, for each type we selected a minimal set of properties that (a) help notebook authors and reusers answer the questions in Chapter 1 (e.g., "what is this object?", "how was it produced?", "what does it depend on?") and (b) can be expressed using RO-Crate, PROV, DCAT, and OntoDT without inventing new vocabularies. This yields profiles that are expressive enough to support our research questions, yet light-weight enough to be captured and confirmed within JupyterLab.

Notebook analysis and helpers. Static and dynamic tools flag issues or recover some provenance. noWorkflow captures provenance for Python scripts without modifying code; YesWorkflow recovers workflow structure from annotations [13, 14]. Linters such as Julynter/PyNBlinT catch anti-patterns [2, 28]. These are useful, but they typically do not produce a *portable research object* with cell-level relationships and a human-navigable graph.

What a user can *do* with an RO-Crate in a Jupyter context. In practical terms, a crate lets a notebook author (or a reviewer) *carry* the story of a result alongside the code:

1. Package the notebook, per-cell scripts (`cells/cell_*.py`), and a machine-actionable description of cell → variable relations (`ro-crate-metadata.json` + a graph export used by the offline viewer).
2. Open the crate locally: double-click a self-contained HTML (`cell_graph.html`) to browse the dependency graph; hover a node to see the code snippet and metadata.

Object	**Entity type(s)**	**Key fields (examples / vocab)**
Notebook	`CreativeWork`	*name*, *description*, *version*, *author*, *license*; *encodingFormat*=`application/x-ipynb+json`.
Cell	`prov:Activity`	*name* ("Cell 7"); *programmingLanguage*; *text* (snippet); links via `prov:used`/`prov:wasGeneratedBy`.
Variable / symbol	`Dataset` (logical)	*name* (symbol), optional *description*; provenance via `prov:wasGeneratedBy`/`prov:used`.
Parameter / config	`PropertyValue`	*name*, *value*; link to the cell (`prov:used`) that consumes it.
File / artifact	`File`	*name*, *contentSize*, *checksum*, *encodingFormat*; provenance as above.
Dataset (curated)	`Dataset`	*name*, *description*, *distribution* (DCAT); *license*, *identifier/DOI*.
Figure / table	`File` or `ImageObject`	*name*, *encodingFormat* (e.g., PNG/SVG, CSV), *about*/*keywords*.
Environment / kernel	`SoftwareApplication`	*name*, *version*, *softwareRequirements* (e.g., image tag, packages).
Component	`SoftwareApplication` + `prov:Activity`	*name*, *identifier/URI*, *inputs*/*outputs*; link back to notebook cells.
Workflow (.naavrewf)	`CreativeWork` (additionalType=`NaaVREWorkflow`)	*name*, *description*, *encodingFormat* (JSON/YAML), *identifier/URI*; links to components.
Metadata record / Crate	`CreativeWork`	*name*, *license*, *creator*; crate graph (`ro-crate-metadata.json`).

Table 2.2: Compact per-type profiles using RO-Crate, PROV, and DCAT. Acquisition details appear in Chapter 4.

3. Exchange the crate: upload to a registry (e.g., SEEK item or WorkflowHub attachment) or archive (e.g., Zenodo); downstream tools can index JSON-LD, and humans can read the graph without services.
4. Reuse: search for "cells that produce `df_clean`", or "cells that consume `rainfall_data`" (locally or, later, in a triple store), and copy only the needed, well-explained parts.

This is complementary to execution platforms: it focuses on *explainability and portability* of relationships, not just re-running.

Object lifecycles (conceptual). We refer to lightweight lifecycles that reflect notebook practice; Chapter 4 explains how CellScope captures and packages the associated metadata.

- **Cell** — *author → (optional) execute → analyse (derive defs/uses, I/O) → package as Activity → reuse (copy/refactor or containerize).*
- **Variable/symbol** — *defined → used/derived → packaged as logical Dataset with provenance → referenced downstream.*
- **File/artifact / Figure/Table** — *produced → consumed/published → versioned → archived*; captured via `prov:wasGeneratedBy`/`prov:used`.

- **Notebook** — *draft* → *analysed* → *packaged as RO-Crate* → *shared/archived* (registry/DOI) [11, 12].
- **Environment/kernel** — *selected* → *recorded* (image, packages) → *replayed* (optional).
- **Component** — *derived from cells* (containerization) → *published to VRE* → *invoked/composed* [10].

These lifecycles frame the evaluation criteria used later (coverage of defs/uses/edges, portability of packaged objects, and usefulness of the graph/list for discovery).

2.2 Related Work

Method. We followed a lightweight literature study. Sources: Google Scholar, ACM Digital Library, IEEE Xplore, and project/documentation sites referenced by primary works. Core search strings included: *"RO-Crate specification 1.1"*, *"workflow registry WorkflowHub"*, *"Galaxy update 2024"*, *"Jupyter provenance"*, *"noWorkflow"*, *"YesWorkflow"*, *"ProvBook"*, and *"CF Conventions"*. We screened titles/abstracts and prioritised: (i) peer-reviewed or official specs; (ii) practical systems used by communities; (iii) materials that explicitly address packaging, provenance, or notebook/script analysis. We included recent updates where available (e.g., Galaxy 2024 and WorkflowHub 2024/2025). Where tools were primarily described in documentation rather than peer-reviewed papers, we treated official specifications and project documentation as primary sources and used academic publications mainly to contextualise scope and evaluation claims. Search was conducted between April and June of 2025

Packaging/specifications. *RO-Crate* provides a web-native, JSON-LD based packaging format to colocate files and machine-actionable metadata; version 1.1 is the current stable specification. It emphasises schema.org alignment and extensibility, and offers workflow/run profiles to describe inputs, outputs, parameters, and execution traces [11, 12, 22]. This is the backbone we target for portable "notebook + context" packages.

PROV-O (W3C) and *DCAT v2* (W3C) are key vocabularies we reuse: PROV for *wasGeneratedBy/used/wasDerivedFrom*, DCAT for dataset/catalog semantics [5, 20, 21]. These specifications are intentionally general; Chapter 3 explains how we specialise them for cell-level digital objects in notebooks.

VRE registries and execution platforms. *WorkflowHub* serves as a registry for FAIR computational workflows (with updated lifecycle publications through 2025). It focuses on workflow-level descriptions and lifecycle management across engines, supporting discovery and citation of reusable workflows [17, 29]. *Galaxy* is a mature web platform for accessible, reproducible analysis, with rich support for tool definitions, histories, and workflow construction in a web UI [23, 30]. *FAIRDOM-SEEK* excels at managing datasets, models, and SOPs with rich metadata and DOIs, supporting sharing and long-term preservation of research assets [18]. *Describo* (RO-Crate editor) helps curate RO-Crates for arbitrary research objects, offering guided metadata entry and validation for crates [31]. *nf-core/Nextflow* communities provide high-quality, versioned pipelines and a rich ecosystem of best-practice workflows for production settings. We later analyse in Section 2.4 how these platforms relate to our research questions around cell-level metadata for digital objects.

Notebook/script provenance and analysis. *noWorkflow* collects provenance for Python scripts without modifying code, with strong support for execution-time lineage queries and a provenance store [13]. *YesWorkflow* exposes prospective provenance (workflow structure and dataflow) from scripts via lightweight, language-agnostic annotations in comments, and supports graph visualisation of the inferred workflow [32]. *ProvBook* captures and visualises Jupyter execution provenance (e.g., cell runs) and can export RDF for further analysis [33, 34]. These systems collectively inform what kinds of provenance information are useful in practice; Section 2.4 revisits how they align with our research questions.

Domain conventions and semantics. For climate notebooks, the *CF Conventions* and *CF Standard Names* underpin interoperable data/metadata practice. They matter when we lift our crates into domain search or validation. [26, 27]

Illustrative reproduction paths. *WorkflowHub:* publish a workflow; to replay a notebook, you must refactor cells into a workflow description. Pitfall: loss of interactive, exploratory context; no cell-level lineage unless separately encoded. *FAIRDOM-SEEK:* upload notebook, datasets, and descriptive

metadata for sharing/DOI. Pitfall: no automatic extraction of cell relations; consumers still guess "which cell produced what". *Galaxy:* wrap each step as a tool or use an interactive environment. Pitfall: tool-level provenance but not notebook cell graph; manual effort to reach runnable parity. *nf-core/Nextflow:* re-implement as a pipeline. Pitfall: great for production, but requires rewriting; loses the "as-authored" notebook structure. *Describo:* curate an RO-Crate by hand. Pitfall: manual metadata entry; no static analysis to derive the dependency graph. These examples illustrate how researchers currently move from notebooks to more structured artefacts; Section 2.4 returns to how well these paths support cell-level metadata for digital objects.

What these works provide for us. In short: RO-Crate/PROV/DCAT give us the *interchange language*; WorkflowHub/Galaxy and related VREs show FAIR packaging and registry integration at the workflow or asset level; noWorkflow/YesWorkflow/ProvBook show how provenance can be captured or exposed for scripts and notebooks. The missing piece, and our contribution, is a cell-level metadata layer for digital objects inside Jupyter that covers how code cells, variables, and files relate and under which conditions they were produced, and that is packaged as RO-Crate and exposed via an interactive, developer-friendly graph and list view.

Classification (summary)

A summary of the researched work and their respective categories can be found in table 2.3.

Category	Representative work
Packaging/specs	RO-Crate core/spec [11, 12], PROV-O [5], DCAT v2 [21], Describo [31]
VRE registries/execution	WorkflowHub [17, 29], Galaxy [23, 30], FAIRDOM-SEEK [18], **NaaVRE** [10], nf-core/Nextflow
Notebook/script provenance	noWorkflow [13], YesWorkflow [32], ProvBook [33, 34]
Notebook quality/linters	Empirical studies and tooling [1, 2, 28]
Domain conventions	CF Conventions [26], CF Standard Names [27]
Visualisation building blocks	NetworkX [15], vis.js (Network) [16]

Table 2.3: Related work classification used in this thesis.

2.3 Existing solutions

We now compare a small set of concrete systems and tools that are closest to CellScope's goals, focusing on practical differences in granularity, capture mode, portability, and Jupyter user experience. Table 2.4 provides a head-to-head summary; the paragraphs that follow briefly describe what each tool has, and what it does not directly address, in the context of our research questions.

Per-tool summary.

ProvBook. ProvBook extends Jupyter notebooks with execution provenance: it records cell runs, tracks how outputs change over time, and can export provenance information in RDF [33, 34]. This directly informs what kinds of runtime context are useful to retain for notebooks. However, ProvBook does not define a portable research object focused on digital objects (files, variables, figures) and their metadata, nor does it provide a separate, notebook-agnostic package such as an RO-Crate for reuse outside the original environment.

noWorkflow. noWorkflow captures detailed execution provenance for Python programs without requiring code modifications, storing function calls, variable values, and dependencies for later querying [13]. It demonstrates the value of rich provenance stores and query interfaces. Its primary focus, however, is on script-level execution histories rather than cell-level digital objects in notebooks, and it does not prescribe a RO-Crate-style bundle that travels with a notebook.

YesWorkflow. YesWorkflow reveals prospective provenance from scripts by interpreting special annotations in comments, producing dataflow graphs and documentation [32]. It shows how lightweight annotations can turn informal scripts into explicit workflows. At the same time, it assumes annotated scripts rather than unannotated notebook cells, and it does not target an integrated JupyterLab experience or a per-notebook metadata package.

WorkflowHub. WorkflowHub acts as a registry and sharing platform for FAIR computational workflows, enabling discovery, DOIs, and metadata-rich records across workflow engines [17, 29]. It is strong at the workflow level and at lifecycle management, but it expects workflows in dedicated languages (e.g., CWL, Nextflow) rather than as-authored notebooks. Cell-level metadata about digital objects inside notebooks is therefore outside its scope.

Galaxy. Galaxy is a web platform for building, executing, and sharing analyses through tools and workflows, with detailed histories of runs and parameters [23, 30]. It excels at pipeline-style reproducibility and user-friendly execution. Jupyter notebooks can be used alongside Galaxy (e.g., via interactive environments), but the platform does not model the notebook's internal digital objects and their metadata as a portable package in the way we require for CellScope.

FAIRDOM-SEEK. FAIRDOM-SEEK manages datasets, models, and standard operating procedures with rich metadata and DOIs, helping researchers share and curate assets over time [18]. Notebooks and files can be uploaded as assets, but intra-notebook relationships between cells, variables, and files remain implicit unless the author encodes them manually; SEEK does not provide automatic or structured cell-level metadata for notebook digital objects.

Describo. Describo is an editor for creating and maintaining RO-Crates, offering guided metadata entry and validation for arbitrary research objects [31]. It is well suited to manual curation of crates, including crates that might contain notebooks. What it does not attempt is to derive metadata from notebook code itself; it expects the user to supply metadata fields rather than extracting a cell-level view of digital objects.

NaaVRE Component Containerizer and analysers. The NaaVRE Component Containerizer builds reusable components from notebook steps, supported by a Code analyser / I/O Detector and a Type Detector that infer inputs/outputs and simple types for cells at containerisation time [10, 19]. This is effective for turning notebook fragments into pipeline components and wiring them into workflows. However, the analysers are oriented towards container specs and workflow composition, not towards producing a persistent, notebook-wide metadata record of digital objects and their relationships that can be packaged and exchanged independently of the execution environment.

Takeaways. Tools closest in spirit either (i) emphasise *runtime* histories and execution stores (ProvBook, noWorkflow), (ii) rely on *annotations* to recover workflow structure (YesWorkflow), or (iii) operate at *workflow/platform* or asset scope (WorkflowHub, Galaxy, FAIRDOM-SEEK, Describo, NaaVRE) rather than at the level of notebook digital objects. None of them combine a cell-level metadata model for notebook digital objects, low-friction capture with confirm-first curation, and RO-Crate + PROV packaging with in-place JupyterLab views. This combination motivates CellScope's design, which is developed in Chapters 3 and 4.

2.4 Gap Analysis

This section connects the related work to the research questions from Chapter 1. Table 2.5 gives a high-level view of how different categories of work support, partially support, or do not address each research question.

For **RQ1** (which kinds of cell-level metadata and contextual information about digital objects are needed), provenance systems such as noWorkflow and ProvBook illustrate the value of recording executions, dependencies, and parameter settings, while domain conventions like CF and OntoDT show which fields matter for specific scientific communities [13, 25, 26, 33]. However, these works either focus on scripts rather than notebook cells, or on domain-level metadata rather than a cross-domain, notebook-oriented model of digital objects (cells, variables, files, figures) and their context.

For **RQ2** (how to structure and package this metadata in a portable, web-native representation), RO-Crate, PROV, and DCAT already provide a strong foundation for representing research objects and their provenance in JSON-LD [5, 11, 21]. WorkflowHub, Galaxy, FAIRDOM-SEEK, and Describo demonstrate how such descriptions can be registered, shared, and curated in larger ecosystems [17, 18, 23, 31]. What is missing is a concrete, opinionated profile for "notebook + cell-level metadata about its digital objects" that can be generated from JupyterLab and carried as a single, portable package.

For **RQ3** (which interface helps users see and navigate this metadata inside JupyterLab), ProvBook and YesWorkflow show how graphs and timelines can help users explore provenance, while notebook linters provide in-place feedback inside Jupyter [28, 32, 34]. VRE platforms such as WorkflowHub, Galaxy, FAIRDOM-SEEK, and NaaVRE provide web UIs for workflows, components, and datasets. None of these, however, surface cell-level metadata about notebook digital objects as a first-class, navigable graph and list view *inside* JupyterLab itself.

Summarising, the literature and existing systems provide:

- a rich vocabulary and packaging substrate for research objects (RO-Crate + PROV + DCAT),
- examples of provenance capture and visualisation for scripts and notebooks (noWorkflow, YesWorkflow, ProvBook),
- and mature platforms for workflows, datasets, and components (WorkflowHub, Galaxy, FAIRDOM-SEEK, NaaVRE).

The remaining gap, which CellScope addresses in the following chapters, is a JupyterLab-first tool that (i) defines a compact, cell-level metadata model for notebook digital objects, (ii) packages this metadata as a portable research object using RO-Crate and related vocabularies, and (iii) exposes it through integrated graph and list views to help users understand, reproduce, and reuse results.

Tool / System	Granularity	Capture mode	Packaging / Portability	Jupyter UX	Notes / Limitations
ProvBook [33, 34]	Notebook / cell (exec)	Runtime provenance (execution history)	RDF export possible; not RO-Crate-native	In-notebook visualisation	Focuses on execution traces; limited static data-dependence; no portable research object by default.
noWorkflow [13]	Script / cell (exec)	Runtime provenance without code mods	Provenance store; not RO-Crate-native	External UI; not Jupyter-first	Strong execution lineage; less cell-aware for live notebooks; export requires mapping.
YesWorkflow [32]	Script-level	Annotations (prospective provenance)	Graphs / docs; no RO-Crate	External visualisation	Requires user annotations; targets scripts rather than live notebook cells.
WorkflowHub [17, 29]	Workflow	Registry / metadata	FAIR workflow records; attachments	External web UI	Discovery / citation over workflows; replaying notebooks requires refactoring to a workflow language.
Galaxy [23, 30]	Tool / workflow	Execution + histories	Platform internal; not RO-Crate	Web platform	Rich pipeline provenance; notebooks reproduced via tools / IEs; no cell-level lineage as a portable object.
FAIRDOM-SEEK [18]	Dataset / model / asset	Curation (manual)	Rich metadata + DOIs	Web portal	Hosts "notebook + files", but intra-notebook relations remain implicit unless encoded separately.
Describo [31]	Any research object	Manual curation	RO-Crate editor	Desktop / web app	Helpful for curation; does not derive cell relations automatically.
NaaVRE Component Containerizer [10, 19]	Cell / component	Static I/O detection (+type hints)	Container specs; not RO-Crate-wide	Jupyter extensions environment	Effective for containerisation; currently no notebook-wide, portable cell→data provenance package.
CellScope (this work)	Cell + file + variable	Static first	RO-Crate 1.1 + PROV (portable)	Jupyter-first (graph + list)	Automatic cell-level relationships; confirm-first metadata capture; generate→then visualise; VRE-agnostic.

Table 2.4: Comparison of alternatives closest to our goals (cell-level understanding, portability, and Jupyter integration). CellScope row shown for context.

Work / category	**RQ1**	**RQ2**	**RQ3**
RO-Crate + PROV + DCAT	n/a	yes	n/a
WorkflowHub / Galaxy / FAIRDOM-SEEK	partial	yes	partial
noWorkflow / YesWorkflow / ProvBook	yes	partial	partial
Notebook linters (Julynter, PyNBlinT)	partial	n/a	partial
NaaVRE analysers (component containerizer, I/O detector)	partial	n/a	partial
Domain conventions (CF, OntoDT)	yes	n/a	n/a

Table 2.5: How related work informs, but does not fully answer, RQ1–RQ3. "yes" = directly addresses the question; "partial" = informs design at a different granularity or scope; "n/a" = not a primary focus.

Chapter 3

CellScope: A Human-Centered Approach

To address the gap identified in Chapter 2, CellScope is designed around a simple idea: *make notebook internals visible and portable for people and machines*. This chapter outlines methodology, requirements, and a technology-agnostic architecture; implementation details are reserved for Chapter 4.

3.1 Methodology

We adopt a design science research (DSR) approach [35] to build and evaluate an artifact that addresses the gap identified in Chapter 2: the lack of cell-level metadata and contextual information about digital objects inside Jupyter notebooks that is both portable and easy to inspect. The DSR cycle operationalises this gap and the research questions from Section 1.3 into concrete design and evaluation steps. The cycle and its instantiations in this thesis are:

1. **Problem framing and context.** Ground the work in observed notebook pain points (lost context, hidden state, brittle sharing) and evidence from empirical studies [1–3]. Clarify the VRE setting and intended users at a conceptual level (Ch. 1); NaaVRE is used as an illustrative integration example rather than a required dependency.
2. **Requirement elicitation and prioritisation.** Derive candidate requirements from the gap analysis (Ch. 2), supervisor feedback, and a small user-story survey. Prioritise with a lightweight scheme that emphasises *relevance* and *value* (MoSCoW variant), producing R1–R4 (capture, packaging, views, polyglot readiness) plus non-functional goals (portability, findability, near-real-time feedback).
3. **Design principles and conceptual architecture.** Formulate principles that guide the solution: *storage-first*, *confirm-first* metadata (draft → review → store), and *generate then visualise*. Consolidate these into the conceptual architecture and lifecycle in §3.3.
4. **Build minimal instantiations.** Realise the principles in a small, Jupyter-first artifact that (i) captures a draft of intra-notebook relationships, (ii) enables author confirmation, (iii) packages the result as a portable research object, and (iv) renders views that read from what is stored (Ch. 4). We adopt a *static-first* capture stance for safety and speed.
5. **Evaluation and iteration.** Specify objectives, baselines, tasks, KPIs, and analyses in advance (Ch. 5). We evaluate along *coverage* (does capture reflect the intended relations?), *comprehension/efficiency* (do people answer provenance questions faster and as accurately?), and *portability/machine-actionability* (is the package machine-actionable and reusable) [36, 37]. Results inform iterative refinements.
6. **Reflection and communication.** Document the design trade-offs (e.g., profiles vs. single schema; confirm-first vs. increased automation), report limitations, and release artifacts to support reuse and scrutiny.

This cycle is grounded in the gaps identified in Chapter 2. The problem framing and requirements (steps 1–2) respond to the lack of cell-level metadata and portable, web-native packaging for notebook digital objects (RQ1–RQ2). The design and build steps (3–4) realise a concrete artifact, CellScope, that operationalises storage-first, confirm-first metadata capture and dual graph/list views inside JupyterLab

(RQ2–RQ3). Finally, the evaluation and reflection steps (5–6) map onto the evaluation objectives introduced in Chapter 5 (coverage, comprehension/efficiency, and near-real-time behaviour), closing the loop between the gap analysis, research questions, and the implemented tool.

3.2 Requirement Analysis

Elicitation & prioritisation. Requirements were collected from the literature and gap analysis (Ch. 2), and validated and refined through a small user-story survey (4 respondents: 3 NaaVRE developers, 1 non-developer user; all experienced Jupyter users). Participants rated each story's *clarity*, *relevance* and *value* from 1 to 5 and provided free-text feedback. Given the small sample, the survey is used to refine priorities and phrasing rather than to claim statistical generalisation.

To prioritise, we compute a *weighted score* per story:

$$\text{Score} = 0.45 \cdot \text{Relevance} + 0.45 \cdot \text{Value} + 0.10 \cdot \text{Clarity}$$

and map the score to MoSCoW categories with fixed thresholds: *MUST* (≥ 4.5), *SHOULD* (≥ 4.0 and < 4.5), *COULD* (≥ 3.0 and < 4.0), *WON'T (for now)* (< 3.0). This gives more weight to *relevance* and *value*—which better reflect user impact—while still considering *clarity*.

Must-have functionalities (weighted average ≥ 4.5)

- **Filter by type/tags/relations** (precise discovery)

Should-have functionalities ($4.0 \leq$ weighted average < 4.5)

- **Link digital objects across projects**
- **FAIR feedback** on completeness when publishing
- **Publish selected objects** to a marketplace
- **Jump** from a marketplace card to the code+graph

Could-have functionalities ($3.0 \leq$ weighted average < 4.0)

- Increase automation of contextual metadata capture (lower burden)
- See cell dependencies
- Explore graph
- Search for cells/data snippets to reuse

Won't-have functionalities (weighted average < 3.0)

- Advanced ontology-level reasoning beyond lightweight typing

Additional feedback to highlight:

- the need for a *non-graph view* (table/filter) alongside the graph,
- *compatibility cues* (IDs, versions) to avoid breakage,
- and *credit/citation* for reuse.

These results directly inform the requirements and prioritisation below.

Capture (R1). Per cell: capture definitions and uses, plus materialised artifacts such as files; derive edges from last definition to subsequent uses. Prefer static analysis to avoid executing user code [13].

Package (R2). Represent *processes* (cells) and *data* (symbols, files, referenced resources) with explicit input, output, and generation links in a portable package that is human-readable and machine-actionable. Include identity and lightweight version cues to support cross-project linking and compatibility checks.

Explore (R3). Provide two complementary views: a graph for "structure at a glance" (with short rationales on edges) and a compact *non-graph list with filters* (type/tags/relations) for precise discovery. Both views read from the same stored description.

Scope (R4). Python first, with a polyglot strategy based on recognising common hand-offs between languages and kernels when exchanges are made explicit (for example via files). Marketplace publishing and FAIR feedback are tracked as "Should-have" features contingent on the core capture and packaging path.

3.3 Architecture

Purpose. This section explains *what* CellScope is conceptually and *how it is expected to behave* from a user and system point of view, without committing to specific technologies (details are in Chapter 4).

Concept at a glance

CellScope makes the internal structure of a notebook *visible* and *portable*. It does so by:

1. **Capturing and storing** cell-level metadata and contextual information about digital objects (e.g., cells, symbols, files, configuration artifacts), including how they are produced and used across cells (R1).
2. **Packaging** the notebook together with that contextual metadata into a shareable research object that carries provenance and reuse-relevant context (R2).
3. **Exposing** the result through two complementary views: a structural graph and a non-graph list/filter view for precise discovery (R3).
4. **Accommodating multiple languages** by recognising explicit hand-off patterns between cells and kernels, especially when exchanges are materialised as files or other explicit artifacts (R4).

Actors and interactions

- **Author** works in a notebook; asks CellScope to *analyse* and/or *export*.
- **Reviewer/Collaborator** opens the overview to understand "what depends on what," or opens a packaged artifact received from someone else.
- **Registry/Marketplace** (optional) can index packaged artifacts to support discovery and reuse.

Use cases and intended scope

To make the scope concrete, we define use cases that represent how CellScope is intended to be used in practice. They serve as a target for the design in this chapter and as a reference point for the reflection in Chapter 6.

UC1: VRE integrator (platform setup and personalisation). A VRE maintainer deploys CellScope as an optional capability inside their notebook environment. The core tool is VRE-agnostic: it does not require platform-specific metadata. Instead, the integrator configures which digital object types should be recognised and which metadata fields should be requested or recommended (e.g., to align with local governance, domain practices, or publishing constraints). As an illustrative example, a VRE such as NaaVRE may treat workflow definition files as first-class digital objects and may provide project *flavours* that bundle environment descriptors (e.g., dependency manifests, kernel specifications) so exports include reproducibility-relevant context without bundling the runtime itself.

UC2: Virtual lab developer (understanding impact and preventing breakage). A developer maintains a shared notebook-based analysis and needs to safely change it over time. Typical tasks include: checking whether a symbol or intermediate file is still used before removing it; understanding which results may change when modifying a configuration artifact; and locating the exact inputs and parameters behind a figure or table. CellScope supports these tasks by presenting digital objects as first-class entities and by exposing how they are connected across cells.

UC3: External collaborator (receiving an exported package). A collaborator receives an exported research object and wants to reproduce or adapt results without access to the original VRE. They inspect the package locally (graph and list/filter views) to understand what digital objects exist and how they were produced, then use the included context to resolve required inputs (including referenced external resources) and reconstruct a compatible environment using included environment

Group	Fields (examples)
Identity & location	cell index; stable cell id; notebook reference
Execution context	kernel/language; cell type (code/markdown); optional execution count
Content snapshot	code/text snippet; content hash; exported per-cell script file (`cells/cell_*.py`)
Derived structure	`var_defs`; `var_uses`; function defs/calls (when available); inferred producer→consumer edges
Artifact context	file reads/writes (path when resolvable); materialised artifacts packaged in the crate
Provenance links	`prov:used`, `prov:wasGeneratedBy`, `prov:wasDerivedFrom` (as applicable)
User-confirmed hints	roles/tags; optional descriptions; project/domain fields via "flavors"

Table 3.1: Cell-level metadata captured and exported by CellScope (grouped for readability).

descriptor files (when provided). This use case prioritises inspectability and contextual clarity first, and full rerunnability when inputs and environment can be reconstructed.

Storage-first principle. We treat storage as first-class. Every digital object is handled in three parts: its *content* (the bytes), its *metadata* (a concise description that people and machines can understand), and its *control & links* (identity, lightweight versioning cues, provenance relations, and access cues). The author keeps a portable package of metadata co-located with the content; the platform may maintain a separate index optimised for discovery and cross-object linking. This separation preserves portability for authors while making objects easier to find across projects.

Common core with per-type profiles. To balance consistency and expressiveness, we use a small *common core* (identifier, type, name, license, version cue, checksum cue, sensitivity cue) and *per-type* profiles (Notebook, Cell/Activity, File/Dataset, Figure/Table, Parameter/Config, Workflow). The common core enables uniform handling and validation; the profiles capture what is specific to each object without overloading a single schema.

Cell-level metadata This paragraph makes explicit the metadata fields CellScope defines at the level of notebook cells. We group fields into: (i) intrinsic cell properties, (ii) derived structure from static analysis, (iii) artifact I/O context, and (iv) optional user-confirmed enrichment via confirm-first. These metadata fields can be found in table 3.1

The intent is to keep the default schema lightweight and cross-domain, while allowing integrators to require additional domain-specific fields through project-specific "flavors" (Chapter 6 and Chapter 8).

Confirm-first metadata lifecycle. CellScope follows a confirm-first flow: *(1) capture* a draft description from the notebook and its context; *(2) review/confirm* a minimal set of fields where automation is unreliable (e.g., roles, tags, dataset semantics, access cues); *(3) store* by writing an author-side package and (optionally) updating an index; *(4) visualise or export from what is stored.* This prioritises correctness of stored information over presentation.

Automation spectrum (static, manual, and future LLM support). CellScope deliberately mixes (i) *static capture* of structural relations (safe, fast, repeatable), with (ii) *small manual confirmations* for meaning-heavy fields (roles, semantics, access metadata). Large-language models (LLMs) could further reduce manual effort by proposing tags, summarising cell intent, and extracting contextual metadata from surrounding text and docs; however, LLM-assisted capture is treated as future work and is not implemented in the current prototype.

Conceptual components

Capture. Interprets each cell to identify the things it defines (symbols, materialised artifacts) and the things it uses, then infers relationships between cells. The capture is designed to be fast, safe, and minimally intrusive (no changes to user code), acknowledging known fragilities of notebooks (R1).

Packaging. Turns the notebook plus its relationships into a portable bundle with explicit provenance so others—and tools—can inspect it outside the original environment (R2).

Views. Two complementary lenses over the same relationships: (i) a graph for "structure at a glance" and (ii) a list/filter view for precise discovery by type, tags, or relation (R3).

3.3.1 Persistent identifiers (PIDs) for published RO-Crates

CellScope generates an RO-Crate locally and treats the crate as the source of truth. In the current prototype, CellScope does not mint persistent identifiers automatically; instead, a PID is typically obtained when the crate is published to an external repository or registry. A practical default is DOI minting via a research-data repository (commonly backed by DataCite), after which the assigned identifier can be recorded back into the RO-Crate (e.g., `identifier`) for citation and discovery. In a VRE deployment, the same pattern can integrate with an institutional PID service (e.g., Handle or ARK) while preserving the crate-first workflow.

3.3.2 End-to-end workflow (high-level)

Before detailing the architecture, we summarise the end-to-end flow at a conceptual level, focusing on how a digital object (DO) is derived from notebook work and materialised into a portable Research Object (RO) package.

1. **Authoring:** A notebook is created/edited in Jupyter. Digital objects exist initially as implicit entities: cells, symbols, parameters, and referenced or produced files.
2. **Analyse (derive structure):** CellScope derives a cell-level structure view (e.g., definitions/uses, file reads/writes, producer→consumer relations) from the notebook.
3. **Confirm-first (curate semantics):** The user optionally confirms and enriches metadata for selected objects (e.g., titles/descriptions, roles, dataset descriptors). This is where project-specific metadata can be applied.
4. **Materialise artefacts:** Referenced or produced files are collected (when available), and lightweight per-cell snapshots are written to disk so that the package remains inspectable without platform services.
5. **Create the RO:** A portable RO-Crate is generated that bundles files and JSON-LD metadata describing the notebook, cells (as activities), and digital objects (as entities) with provenance links.
6. **Identify and store:** The RO-Crate is stored locally as the source of truth. If published to a repository or registry, a persistent identifier (PID) can be minted externally and recorded back into the crate.
7. **Optional discovery:** For cross-project search, the RO-Crate can be indexed into a triple store as a projection; the crate remains the canonical package.

Typical flow (happy path)

1. The author runs *Analyse*. CellScope builds a conceptual map of cells and their inputs/outputs.
2. The author explores the result: follows upstream/downstream, filters by type/tags/relations, and checks short rationales for edges.
3. The author exports a portable package to share with a collaborator or attach to a record/registry.
4. A collaborator opens the package locally and reaches the same views without needing the original environment.

Near-real-time updates. Conceptually, the platform can react to authoring events (e.g., saving or executing work) by refreshing the draft description and proposing updates for confirmation, keeping feedback timely without requiring the author to manage operational details. The current prototype focuses on explicit *Analyse* and *Export* actions and uses debouncing where appropriate (implementation details in Chapter 4).

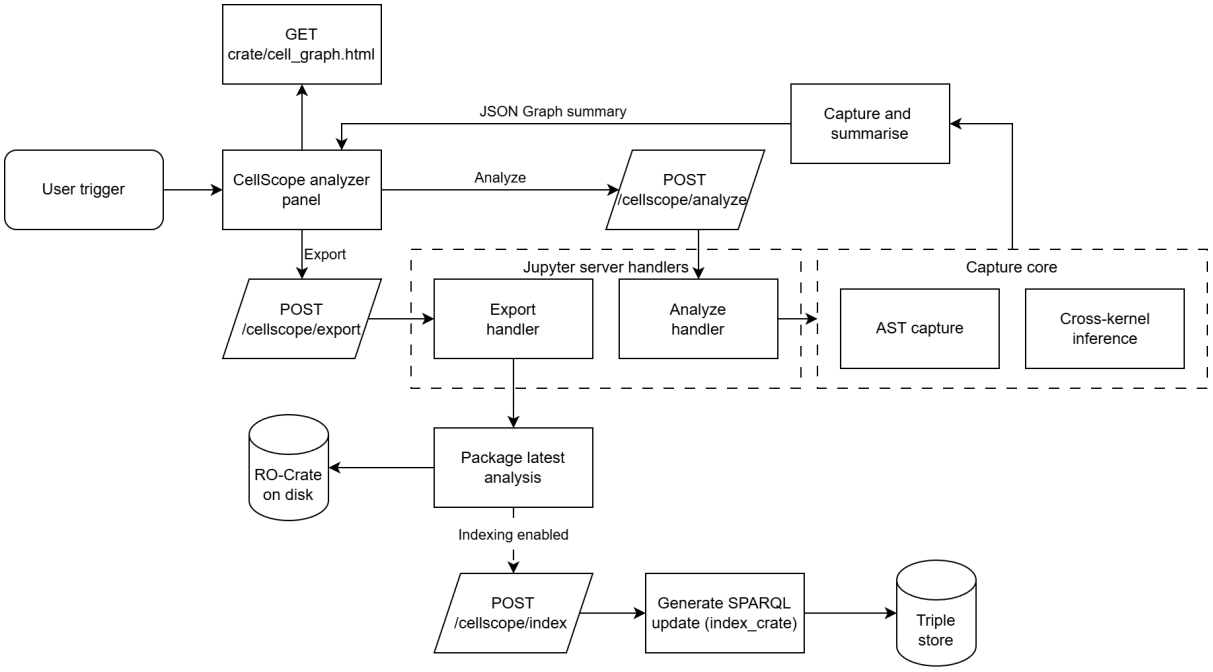


Figure 3.1: High-level analyse and export flow: the author triggers analysis or export from the CellScope panel; the server-side capture core derives cell-level metadata, which is either returned as a JSON graph summary for in-place views or packaged as an RO-Crate and optional index update.

Generate, then visualise. Human-facing views are read-only lenses over stored relationships. We first generate the information and record it, then visualise it from that source.

Implementation note. Concrete technologies, data structures, and APIs that realise this architecture are introduced in Chapter 4, where we detail capture strategies, packaging format, storage/indexing, and Jupyter integration.

Chapter 4

Implementation

This chapter describes the *current prototype* of CellScope and explains how the conceptual architecture from Chapter 3 is realised in software. The implementation is *Jupyter-first* and consists of: (i) a JupyterLab extension (interactive list and graph views, review dialog, export controls), (ii) a Jupyter Server extension (REST endpoints under `/cellscope`), (iii) a reusable Python library (`cellscope`) that performs capture, RO-Crate building, indexing, and visualisation, and (iv) a CLI that mirrors the same pipeline for batch and regression testing.

Importantly, the prototype is *VRE-agnostic*: it can be integrated into any notebook-based environment. NaaVRE is used as an illustrative integration setting (e.g., workflow files and project "flavors"), but the core metadata model and pipeline do not depend on NaaVRE-specific services.

The implementation is released as open-source software [24].

4.1 End-to-end lifecycle

CellScope operationalises the *generate, then visualise* and *confirm-first* principles (Chapter 3, §3.3) using a single shared pipeline, executed either from the UI or from the CLI:

1. **Capture (static).** Parse the notebook and extract per-cell structure (definitions, uses, function defs/calls, file reads/writes) without executing code.
2. **Infer cross-kernel links.** Add edges for file materialisation hand-offs when a file written in one cell is read in another (including across kernels).
3. **Review (optional).** Present a lightweight review dialog where users can assign roles (e.g., `threshold` is a parameter) and annotate file-level metadata (encoding, keywords, access URL, ETag, timestamps). If skipped, defaults are used.
4. **Package as RO-Crate.** Emit an RO-Crate containing: per-cell source snapshots, file entities, environment/config sidecars, JSON-LD metadata, and graph exports (GraphML + HTML).
5. **Index (optional).** Project the crate metadata to RDF triples and emit a SPARQL UPDATE; optionally post it to a configured triple store.
6. **Visualise.** Render views from *stored artifacts* (crate-local graph files) in local mode, or reconstruct summaries from SPARQL in *SPARQL mode*.

The same pipeline is used by: (a) `POST /cellscope/analyse` (capture + inference, return summary JSON), (b) `POST /cellscope/export` (analyse + build crate + optional indexing), and (c) the CLI (`cellscope build/vis`) for headless runs.

4.2 Data model and vocabularies

CellScope stores metadata as RO-Crate JSON-LD [11, 12], using a small set of interoperable vocabularies:

- **schema.org** for general descriptive metadata (`name`, `description`, `encodingFormat`, `keywords`, `version`, `position`, `isPartOf`).
- **PROV-O** for provenance edges (`prov:used`, `prov:wasGeneratedBy`, timestamps) [5].
- **DCAT** for dataset access/distribution metadata when a resource is referenced remotely (e.g., `dcat:accessURL`) [21].

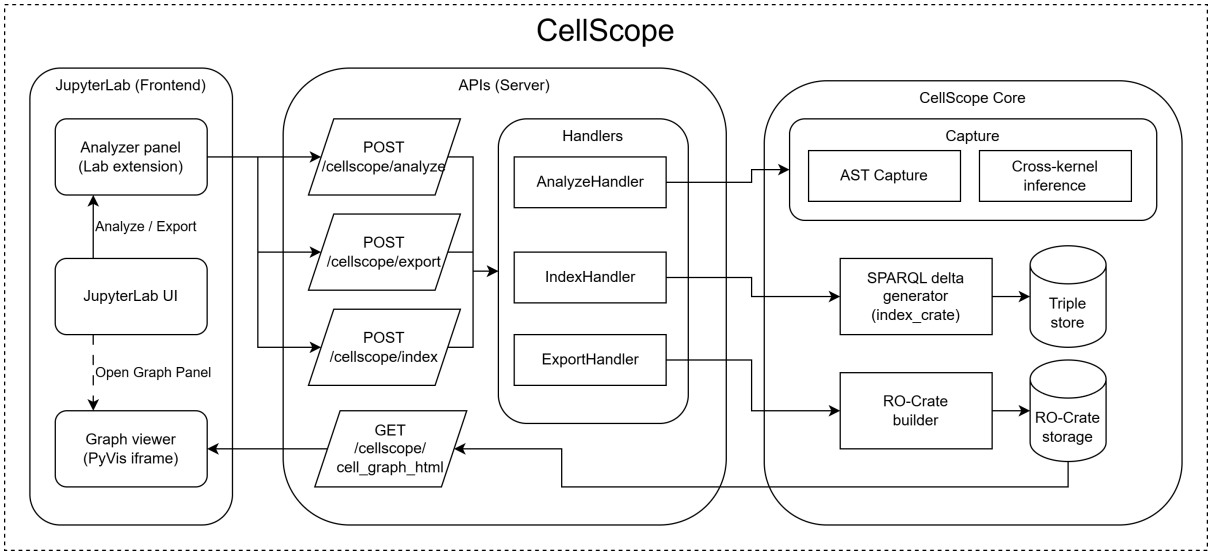


Figure 4.1: Technical architecture of CellScope: JupyterLab frontend, Jupyter server endpoints, core Python pipeline (capture → cross-kernel inference → RO-Crate build → indexing), and optional triple store integration for multi-notebook aggregation.

- **Lightweight activity/data typing** to distinguish notebook activities (cells) from data entities (symbols, files).

Entity mapping (crate graph). The prototype uses a pragmatic mapping that preserves inspectability:

- **Notebook root** as the RO-Crate root dataset (`"./"`).
- **Cells** as both `File` (persisted source snapshot) and `Activity` (provenance step). Each cell is stored as `cells/cell_<idx>.<ext>` (e.g., `.py` / `.R`) and annotated with kernel/language, position, and a short snippet.
- **Symbols** (variables and functions) as internal entities (e.g., `#var-threshold`), linked via `prov:used` and `prov:wasGeneratedBy`.
- **Files/artifacts** as `File` entities under `files/`, with optional metadata for remote provenance (ETag, retrieved time, access URL). Local files may be copied into the crate; missing paths remain as logical entities (to preserve provenance even if the artifact is absent).

This mapping implements R1–R3 in a way that remains portable: the crate can be inspected offline, and the same graph can be rebuilt from JSON-LD or from indexed triples.

4.3 Capture subsystem: static-first extraction

4.3.1 Capture goals and outputs

The capture stage produces a *summary graph* suitable for both UI rendering and crate construction:

- Per cell: kernel/language, label, definitions, uses, function definitions, function calls, file reads/writes.
- Edges: (producer cell → consumer cell) annotated with the symbol(s) or file basename(s) responsible for the dependency.

This implements the conceptual *Capture* component (Chapter 3, §3.3) while respecting notebook safety constraints (no execution).

4.3.2 Python capture (AST-based)

Python cells are parsed using the Python AST. Before parsing, the capture sanitises notebook magics and shell escapes (e.g., lines starting with `%` or `!`) to avoid parse errors. The extractor then collects:

- **Definitions**: assignment targets (including unpacking), function/class names, imports, and other binding forms.
- **Uses**: identifier loads not defined in the same cell.
- **Function calls**: conservative call-site extraction (e.g., `compute_stats(...)`) to enrich UI summaries.
- **File I/O**: heuristics for common libraries and patterns (`open`, `pandas.read_*`, `to_*`, pathlib calls, simple join/concat patterns) to extract materialised paths when they can be resolved statically.

```python
def parse_notebook(nb_path):
    nb = read_ipynb(nb_path)
    cells = []
    last_def = {}   # symbol -> producing cell index
    edges = []

    for idx, cell in enumerate(code_cells(nb)):
        kernel = detect_kernel(cell, nb)          # cell metadata or notebook kernelspec
        label  = label_from_first_comment(cell)  # "# step 1: load data" -> "step_1_load_data"

        code = sanitize_magics(cell.source)
        if kernel.startswith("python"):
            tree = ast.parse(code)
            defs = collect_defs(tree)
            uses = collect_uses(tree) - defs
            funcs = collect_function_defs(tree)
            calls = collect_function_calls(tree, uses)
            reads, writes = detect_file_io(tree, code)

        elif kernel.startswith(("ir", "r")):
            defs, uses, funcs, calls, reads, writes = parse_r_cell_static(code)

        else:
            defs, uses, funcs, calls, reads, writes = set(), set(), set(), set(), set(), set()

        cells.append(CellInfo(idx, label, kernel, defs, uses, funcs, calls, reads, writes))

        # symbol edges: last definition -> this use
        for u in uses:
            if u in last_def:
                edges.append((last_def[u], idx, {"type": "uses", "vars": [u]}))
        for d in defs:
            last_def[d] = idx

    return {"cells": cells, "edges": edges}
```

Listing 4.1: Capture algorithm (simplified): per-cell defs/uses + file I/O + initial edges

4.3.3 R capture (static parser)

For R kernels, the current prototype does *not* rely on an external service. Instead, it uses an in-process static parser/tokenizer that extracts:

- assignments (`<-`, `=`, and rightward forms),
- function definitions (`name <- function(...)`),

- conservative uses (excluding package prefixes `pkg::fn` and member access patterns),
- common I/O calls (`read.csv`, `readRDS`, `write.csv`, `saveRDS`, `download.file`, etc.).

This design keeps the system self-contained (no R runtime integration required on the server side) but inherits the limitations of static parsing for non-standard evaluation and metaprogramming (see §4.12).

4.3.4 Cross-kernel inference via file hand-offs

After per-cell capture, CellScope adds inferred edges for file materialisation hand-offs:

- If a file path (normalised) appears in `writes` of cell A and in `reads` of cell B, add an edge A → B of type `file`.
- These edges enable Python↔R dependencies in multi-kernel notebooks without requiring a polyglot execution framework.

```
def infer_cross_kernel_edges(capture):
    by_write = {}  # norm_path -> producing cell index
    edges = []

    for c in capture["cells"]:
        for p in c.file_writes:
            by_write[norm(p)] = c.idx

    for c in capture["cells"]:
        for p in c.file_reads:
            np = norm(p)
            if np in by_write and by_write[np] != c.idx:
                edges.append((by_write[np], c.idx,
                              {"type": "file", "vars": [basename(np)], "via": "
                                  file"}))
    return edges
```

Listing 4.2: Cross-kernel inference (file hand-off)

4.4 Packaging: RO-Crate builder

4.4.1 Crate contents and on-disk layout

On export, CellScope writes an RO-Crate to a timestamped output directory, typically:

`out-lab/<timestamp>/ro-crate/`

The crate contains:

- **Metadata graph:** `ro-crate-metadata.json` (JSON-LD).
- **Cell snapshots:** `cells/cell_<idx>.py` and/or `cells/cell_<idx>.R`.
- **Artifacts:** `files/` (local copies when available; logical entities otherwise).
- **Environment/config sidecars:** `env/` (dependency manifests and related reproducibility descriptors).
- **Graph exports:** `cell_graph.graphml` and an offline HTML graph viewer `cell_graph.html`.

Environment files: what is and is not included. The crate *does not* bundle the runtime environment itself (e.g., Docker images or full package installations). Instead, it packages *environment descriptors* when available (e.g., `requirements.txt`, `pyproject.toml`, lockfiles, conda YAML) under `env/` and exposes them in metadata as `softwareRequirements`. This supports the UC3 reproduction story: recipients can reconstruct a compatible environment using the descriptors, even when the original VRE runtime is unavailable.

4.4.2 Mapping capture output to JSON-LD entities

During crate construction, the builder:

1. creates one entity per cell snapshot (`File` + activity typing) with `name`, `programmingLanguage`, `position`, `version`, and a short `codeSnippet`,
2. creates one entity per symbol (`#var-<name>`) to serve as a stable node for provenance links,
3. creates one entity per file artifact (`files/<name>`) with `encodingFormat` and optional discovery metadata (`keywords`, `accessURL`, ETag/timestamps),
4. links: defs ⇒ `prov:wasGeneratedBy`/output predicates; uses ⇒ `prov:used`/input predicates.

```python
def build_rocrate(capture, out_dir, xkernel_edges, hints, config_files):
    crate = init_rocrate(out_dir)

    # 1) Root dataset
    root = crate.root_dataset(name=capture["notebook_name"],
                              license=hints.get("license", "CC0-1.0"))

    # 2) Add cell snapshots as File + Activity
    for c in capture["cells"]:
        cell_path = write_cell_snapshot(out_dir, c)  # cells/cell_<idx>.py or .R
        cell_ent = crate.add_entity(cell_path,
            types=["File", "Activity"],
            props={"name": c.label, "programmingLanguage": c.kernel,
                   "position": c.idx, "version": 1,
                   "codeSnippet": snippet(cell_path)})
        root.hasPart.append(cell_ent)

    # 3) Add symbols and connect provenance
    for edge in merge_edges(capture["edges"] + xkernel_edges):
        src_cell = crate.cell_entity(edge.src)
        tgt_cell = crate.cell_entity(edge.tgt)
        for v in edge.vars:
            sym = crate.get_or_create_symbol(f"#var-{v}")
            crate.link_prov(sym, produced_by=src_cell, used_by=tgt_cell)

    # 4) Add file entities (local copies when available)
    for f in all_files_from_capture(capture):
        f_ent = crate.add_file_entity(out_dir, f, hints=hints.get("files", {}))
        root.hasPart.append(f_ent)

    # 5) Add env/config sidecars and parse softwareRequirements
    for cfg in config_files:
        crate.copy_into_env(cfg)
        crate.register_env_descriptor(cfg)
    crate.derive_software_requirements_from_env()

    # 6) Emit GraphML + HTML viewer from the same merged graph
    write_graphml(out_dir, capture, xkernel_edges)
    write_html_graph(out_dir, capture, xkernel_edges)

    crate.write_metadata()
    return out_dir
```

Listing 4.3: RO-Crate build (simplified): entities + provenance links

4.4.3 Remote resources and reproduction cues

When an input is a remote resource (e.g., a dataset referenced by URL), the crate can record it as a file/dataset entity with: `dcat:accessURL` and (when available) metadata such as ETag and Last-Modified timestamps. Optionally, remote artifacts may be downloaded into `files/` depending on deployment

policy and size limits; otherwise the crate remains lightweight and records a resolvable access URL.

This addresses the "how does a recipient reproduce from an exported crate" scenario: the crate carries (i) explicit provenance structure (what was used/generated), (ii) resolvable references for required inputs, and (iii) environment descriptors for reconstructing dependencies.

4.5 Indexing: SPARQL projection and named graph versioning

4.5.1 Why index if the crate already exists?

RO-Crates are the canonical portable package (storage-first). Indexing is a separate projection step to support: *cross-notebook* discovery (search by symbol/file/keyword) and *multi-notebook* aggregation in collaborative settings.

4.5.2 SPARQL update generation

The indexer loads `ro-crate-metadata.json`, walks entities, and emits an RDF triple projection capturing: types, names, positions, versions, encoding formats, keywords/access URLs, and core PROV relations. The update is written to `index/last_update.sparql` and can optionally be POSTed to a configured triple store.

To avoid duplicate graphs, the prototype uses a named-graph scheme:

`https://cellscope.local/graph/<slug>?v=<n>`

where `<slug>` is derived from the notebook name and `<n>` is an export counter. Each export drops and rewrites the named graph.

```
def index_crate(crate_dir, endpoint=None):
    g = graph_uri_for(crate_dir)  # https://cellscope.local/graph/<slug>?v=<n>
    triples = collect_triples_from_rocrate(crate_dir)  # schema + prov + dcat +
        custom terms

    sparql = render_prefixes() + f"""
    DROP SILENT GRAPH <{g}>;
    INSERT DATA {{
      GRAPH <{g}> {{
        {triples}
      }}
    }}
    """
    write_last_update(crate_dir, sparql)  # index/last_update.sparql

    if endpoint is not None:
        post_with_retry(endpoint, sparql)  # bounded retries/backoff/timeouts
```

Listing 4.4: Indexing algorithm (simplified): DROP+INSERT into versioned named graph

4.5.3 Configuration and authentication

Indexing is configured via environment variables or server settings, including: `CELLSCOPE_SPARQL_ENDPOINT`, optional auth (token or user/password), and retry/backoff controls. This keeps the system deployable across VREs with different security models.

4.6 Visualisation: GraphML + offline HTML

The export includes two graph artifacts:

- **GraphML** for downstream tools and headless inspection.
- **Offline HTML graph** for interactive inspection outside the VRE.

Both are rendered from the same merged edge set (AST edges + cross-kernel file edges). The HTML view is designed for review and comprehension: nodes correspond to cells, edges are annotated with the responsible symbol/file label(s), and a side panel shows code snippets and key metadata. The list/filter view in the JupyterLab extension reads from the same graph summary and supports precise discovery by kernel, edge type, roles, and file metadata.

4.7 Server extension: endpoints and modes

The Jupyter Server extension registers endpoints under `/cellscope`. The core endpoints are:

- `POST /cellscope/analyse`: capture + cross-kernel inference; returns a JSON summary (cells, edges, file metadata tokens).
- `POST /cellscope/export`: analyse + build crate + optional indexing; accepts user hints and a list of config/environment files to package.
- `POST /cellscope/index`: index an existing crate from disk (or from supplied JSON-LD).

In addition, the prototype supports a *SPARQL mode* for collaborative aggregation: `/cellscope/sparql_summary` reconstructs the latest graph per notebook from triples (selecting the highest `?v=` version), and `/cellscope/sparql_graph` can render an HTML view from the SPARQL-reconstructed graph. This enables the same UI to operate either: (i) purely locally/offline (crate files), or (ii) against a shared triple store (multi-notebook aggregation).

4.8 JupyterLab extension: user flow and review dialog

The JupyterLab extension provides the primary user experience:

1. **Analyse.** Trigger capture; render a list summary and expose filters.
2. **Review (optional).** A dialog allows assigning *roles* to symbols (e.g., parameter vs. intermediate) and annotating *file metadata* (format, keywords, access URL, ETag, timestamps).
3. **Export.** Build a crate with those hints and optionally push an index update.
4. **Open graph.** Open the crate-local HTML, or request a SPARQL-rendered HTML in SPARQL mode.

Settings (endpoint URL, auth, retry policy, and data source selection) are persisted locally so the extension can operate consistently across sessions.

4.9 CLI utilities, workflow capture, and parity testing

The CLI mirrors the same pipeline:

- `cellscope build <notebook> --out <dir>`: capture + build crate (+ optional index).
- `cellscope vis <crate>`: render HTML if missing.

Workflow-level capture is implemented as an *optional* feature: a workflow description (e.g., `.naavrewf`) can be parsed into nodes and edges, each node resolved to a notebook when possible, and the same per-notebook pipeline executed to produce per-node captures and optional crates. A manifest records node status and crate paths. This supports the "export complete workflows" scenario while keeping the notebook-level use case primary.

To ensure consistency between local and SPARQL aggregation, the prototype includes a parity check that compares the locally captured summary to the SPARQL-reconstructed summary for the same notebook.

4.10 Personalization and extensibility

CellScope includes explicit hooks for VRE-specific personalization without embedding VRE-specific schema into the core:

- **Metadata config:** an environment-driven configuration (`CELLSCOPE_METADATA_CONFIG`) can extend which predicates and fields are stored/indexed.
- **UI review extensions:** adding a new field follows a predictable path: extend the review dialog → pass values as hints → store them on RO-Crate entities → map them to RDF triples in the indexer → add list filters if needed.
- **Environment descriptors:** VRE integrators can standardise which "flavor" files (kernels, requirements, lockfiles) are offered for packaging, aligning export contents with local governance and reproducibility practices.

4.11 Alternatives and design trade-offs

Runtime tracing vs. static parsing. High-fidelity provenance could be obtained by runtime tracing inside kernels, but it increases coupling, overhead, and security risk. The current prototype prioritises safe and explainable extraction via static parsing, augmented with user hints when necessary.

Single schema vs. layered vocabularies. Rather than introducing a bespoke ontology, the prototype composes RO-Crate with PROV/DCAT and lightweight activity/data typing to remain interoperable and queryable.

Local-only vs. indexed collaboration. Local crates maximise portability and offline inspection; SPARQL indexing adds cross-notebook discovery and aggregation. The prototype supports both modes with shared data contracts.

4.12 Limitations

CellScope is static-first, so it under-approximates dynamic behaviours: runtime-computed paths, reflection, non-standard evaluation (notably in R), and opaque library-specific I/O can be missed. Cross-kernel linking is strongest when exchanges are materialised as files. Dense notebooks can yield visually crowded graphs; the UI relies on filtering and search to keep the views usable. Finally, remote dataset reproducibility depends on source stability when payloads are not cached inside the crate.

Chapter 5

Evaluation

We evaluate CellScope on its main promise: make it easier to understand, reproduce, and manage results in Jupyter notebooks, while keeping artifacts portable. We follow two complementary tracks: (i) *design of experiments* that map directly to requirements and contributions, and (ii) clear *KPIs*, procedures, and analyses per experiment.

5.1 Objectives

- **O1 (Coverage).** How completely do we capture cells, variable definitions/uses, and producer→consumer edges under a static analysis model? *(RQ1)*
- **O2 (Comprehension & efficiency).** Do users locate definitions, dependencies, and file-level context faster when navigating a large notebook? *(RQ3)*
- **O3 (Real-time & scale).** Can CellScope analyse and (optionally) index results fast enough for interactive use? *(RQ2/RQ3)*

Objective	Primary KPI(s)	Maps to
O1 Coverage	F1 (defs/uses/edges)	RQ1
O2 Comprehension	Time-to-answer (task durations)	RQ3
O3 Real-time & scale	P50/P95 analyse; P50/P95 analyse+index; SPARQL latency	RQ2/RQ3

Table 5.1: Evaluation objectives mapped to RQs and KPIs.

Why FAIRness is not validated as an objective. This thesis does not evaluate FAIRness scores of exported digital objects as a evaluation objective because FAIRness depends on decisions and infrastructure that are outside the scope of CellScope itself. In our workflow, the RO-Crate is produced locally and may not be published with persistent identifiers, landing pages, repository-level access controls, or community-governed metadata requirements. Tools such as F-UJI and FAIRshake primarily assess FAIRness at the level of *published* digital objects (e.g., datasets with resolvable PIDs and repository metadata endpoints) and would therefore measure the publishing platform and curation choices more than the CellScope capture and packaging pipeline. In addition, these tools are not designed to interpret notebook-internal, cell-level entities as first-class FAIR objects. For these reasons, we focus evaluation on what CellScope directly controls: capture coverage (O1), user navigation efficiency (O2), and interactive performance with optional indexing (O3).

Baselines and conditions. We use two conditions for the user study: **C0** Notebook-only (baseline JupyterLab navigation) and **C1** CellScope (list/filters and graph views after *Analyse*). We do not include runtime-provenance tools as competitors in the user study because their primary goal is execution tracing, while CellScope is designed to support execution-agnostic navigation and packaging.

5.1.1 Operationalisation and hypotheses.

O1 (Coverage). Coverage is evaluated by comparing the static capture output against explicit gold templates for each notebook under `examples/`. For each notebook, predictions are generated by the capture pipeline:

```
parse_notebook(..., collect_materialized=True)
infer_cross_kernel_edges(capture)
capture_to_json(capture)
```

This produces per-cell sets of `var_defs`, `var_uses`, file reads/writes, and inferred producer→consumer edges. Predictions are written to `out-validation/gold_predictions/`. Gold templates are authored for each notebook and stored in `out-validation/gold_templates/`. Two documented rule sets are used: (i) a Python AST rule set (definitions include assignment targets, imports, function/class names, and parameters; uses include name loads not defined in the same cell; file I/O includes literal/derivable paths from common read/write calls), and (ii) an R regex rule set for smaller R notebooks (definitions from assignment targets and function definitions; uses from identifiers in expressions not defined in the same cell; file I/O from common read/write calls when paths are literal or resolvable). For synthetic notebooks, additional manual labels are applied to reflect the intended gold standard.
Gold governance. Gold templates were authored by the project author using documented rules and then manually spot-checked against notebook source. A second annotator also confirmed these gold templates.

- *H1a:* On Python notebooks, CellScope achieves near-perfect coverage for definitions, uses, and inferred producer→consumer edges (F1 ≥ 0.95).
- *H1b:* On mixed-kernel notebooks, cross-kernel dependencies are recovered when they materialise as file hand-offs, with edge F1 ≥ 0.70 on notebooks where the non-Python kernel is correctly detected.

O2 (Comprehension & efficiency). We run a within-subjects user study comparing two conditions (C0 notebook-only vs. C1 CellScope). Participants answer provenance and dependency questions without executing cells. To reduce learning effects, tasks are split into two equivalent task sets (A and B), each containing three tasks. The order of conditions is counterbalanced (C0→C1 vs. C1→C0), and task sets are swapped across conditions so that each condition is paired with both task sets across participants.

The primary outcome is **time-to-answer** (seconds) per task and per condition. We do not use correctness as a primary metric in this study because participants were not scored for accuracy in a consistent rubric during data collection; the analysis therefore focuses on efficiency improvements.

- *H2a:* Participants complete provenance and dependency-navigation tasks faster with CellScope (C1) than with baseline notebook navigation (C0), measured as lower total time-to-answer per condition.
- *H2b:* Post-evaluation usability improvements further reduce time-to-answer with CellScope compared to the initial prototype (i.e., Round 2 C1 < Round 1 C1).

O3 (Real-time & scale). Measure end-to-end latency for *analyse+index* on notebooks of 10/50/100 cells under single-user and scripted multi-user loads. Record P50/P95 latencies and update throughput; time common SPARQL templates (producers, consumers, datasets per notebook).

- *H3a:* For $\leq$50 cells, P95 analyse+index < 400 ms; for $\approx$ 100 cells, P95 < 2 s.
- *H3b:* Median SPARQL query time < 200 ms on the indexed store.
- *H3c:* For notebooks in `examples/`, *analyze* remains interactive (P95 < 200 ms).

KPI-requirement mapping. O1→R1 (capture), O2→R3 (views), O3→R2 (packaging), cross-kernel edges in O1→R4 (polyglot).

5.2 Datasets, baselines, and tasks

Notebooks.

1. **N1: Exhaustive Python notebook** ("exhaustive_python.ipynb"; single-kernel; crafted to stress definitions/uses and dense reuse).
2. **N2: RAVL notebook** ("RAVL_sanitized.ipynb"; large, Python-only for the study; references external services and secrets; used strictly for comprehension without execution).

3. **N3: Multi kernel demo notebook** ("multi_kernel_demo.ipynb"; multi-kernel; demo notebook used throughout development)

Baselines & eligibility. For all objectives, we use N1, N2 and N3. For O1/O3, we use additional real notebooks.

Tasks (O2). Participants are given access to three notebooks (`RAVL_sanitized.ipynb`, `multi_kernel_demo.ipynb`, `exhaustive_python.ipynb`) and are not told which notebook contains the answer for a given prompt.

Task Set A (3 tasks).

- **A.1 Impact/dependency tracing (variable).** Find where `conf_local_radar_db` is defined and name two downstream cells that depend on it.
- **A.2 File handoff (producer→consumer, cross-notebook).** The file `readings.csv` is produced in one notebook and later read in another. Identify the producer cell and the first consumer cell.
- **A.3 Reverse provenance ("what produced this output?").** You see ODIM files under `conf_local_odim`. Identify the producing cell and one key input it depends on.

Task Set B (3 tasks).

- **B.1 Impact/dependency tracing (variable).** Find where `secret_minio_access_key` is defined and name two downstream cells that depend on it.
- **B.2 File read (cross-notebook, read-only).** The file `summary.json` is produced in one notebook and read in another. Identify the cell where it is read.
- **B.3 Reverse provenance ("what produced this output?").** You see VP outputs under `conf_local_vp`. Identify the producing cell and one key input it depends on.

5.3 Participants, design, and procedure

Participants. We recruited $n = 10$ participants with mixed JupyterLab experience (2 beginner, 5 intermediate, 3 advanced). Consent and anonymisation were applied.

Design. We use a within-subjects design with two interface conditions: **C0** Notebook-only (baseline JupyterLab navigation) and **C1** CellScope (Analyse + list/filter + graph). Condition order is counterbalanced (C0→C1 vs. C1→C0). To reduce memory effects, tasks are split into two comparable task sets (A and B), and task sets are swapped across conditions so that each condition is paired with both task sets across participants.

Procedure. Participants are given access to three notebooks and are instructed not to execute any cells. Each condition block consists of three timed tasks (either Task Set A or B). Between condition blocks, participants complete a short reset task. For the CellScope condition, participants receive a brief introduction and then use *Analyse* before starting the timed tasks. The full script, tasks, and answer keys are documented in the study protocol.

Instrumentation. We log start/stop timestamps per task and record short qualitative notes (e.g., navigation strategy and observed friction points). In this iteration, correctness is not used as a primary outcome because a consistent scoring rubric was not applied across all sessions; analysis therefore focuses on time-to-answer.

5.4 Metrics and analysis

Coverage (O1). Coverage is computed by comparing gold vs. prediction *sets*:

- **Defs:** set of $(cell\_idx, symbol)$ pairs from gold defs vs. predicted `var_defs`.
- **Uses:** set of $(cell\_idx, symbol)$ pairs from gold uses vs. predicted `var_uses`.
- **Edges:** set of $(source\_idx, target\_idx, symbol)$ triples from gold edges vs. predicted edges.

Precision, recall, and F1 are computed for defs, uses, and edges for each notebook. Consolidated results are stored in `out-validation/benchmarks/coverage_results_all.md` and `out-validation/benchmarks/coverage_res`

Category	Post-Round-1 change
Findability of search/filter results	Results matching a searched filter are highlighted in the list view; exact matches are pinned at the top of the filter list with contextual metadata (object type, path, and where it is used/defined/read-/written).
Workflow friction (multi-notebook analysis)	`Analyse` prompts for a path and then allows selecting multiple notebooks; notebooks are analysed sequentially.
Export correctness	Fixed an issue where exporting changed the graph to show only the exported notebook; export now includes all analysed notebooks.
Session usability	Fresh sessions automatically load SPARQL results (no initial Analyse click needed).
Deferred to future work	Add confirmation button for the filter popup; improve overall UI aesthetics.

Table 5.2: User-feedback-driven and engineering fixes applied after Round 1, followed by re-evaluation in Round 2.

We additionally summarise dominant error modes (e.g., variable-driven file paths, parameter handling, and kernel identification).

Efficiency and comprehension (O2). We analyse time-to-answer using paired comparisons between C0 and C1. Because the design is within-subjects with non-normal timing distributions, we report median and interquartile range (IQR) per condition and apply a Wilcoxon signed-rank test on per-participant total time (sum of the three tasks in each condition). Observational notes are summarised qualitatively.

Latency and scalability (O3). We benchmark local performance for *Analyse* and *export(+index)* across notebooks in `examples/`. Analyse time is measured as `parse_notebook` + `infer_cross_kernel_edges` + `capture_to_json` (5 runs per notebook). Export(+index) time is measured as `parse_notebook` + RO-Crate build + `index_crate` with no endpoint configured (3 runs), isolating local compute time without network overhead. Results are reported as P50/P95/min/max and stored in `out-validation/benchmarks/benchmark_re` and `out-validation/benchmarks/benchmark_results_examples.json`.

For SPARQL latency, we index all example notebooks into a fresh Fuseki store (empty at start), using a stable `graph_uri` per notebook. We execute fixed query templates (e.g., list graphs, count triples, producers, consumers, datasets-per-graph) 10 times each and record P50/P95/min/max. Results are stored in `out-validation/benchmarks/sparql_latency_loaded_examples.md` and `out-validation/benchmarks/sparql_lat`

5.4.1 Iterative improvement after Round 1 (O2)

We ran the O2 user study in two rounds. Round 1 evaluated an early CellScope prototype and collected usability feedback. Based on the observed friction points, we implemented targeted interface and workflow improvements and then re-validated the tool (Round 2) on the same task structure to quantify the impact of those changes.

5.5 Results

5.5.1 O1: Coverage

Across the notebooks under `examples/`, CellScope achieves perfect or near-perfect coverage on controlled Python notebooks, with recall losses primarily driven by (i) function parameter handling differences between the implemented parser and the gold rule set, (ii) variable-driven file I/O (paths constructed via variables rather than string literals), and (iii) kernel identification gaps for R environments.

Per-notebook coverage. Figure 5.1 summarises F1 for definitions, uses, and inferred producer→consumer edges across the notebooks under `examples/`. Results are perfect on the controlled Python notebook and

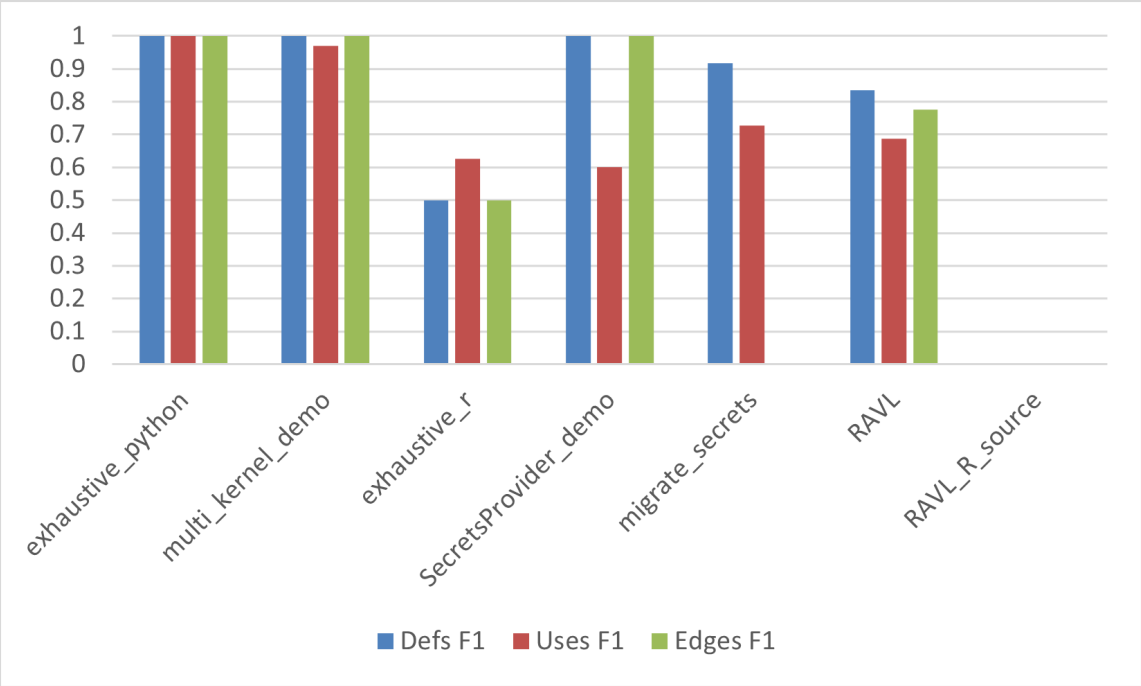


Figure 5.1: O1 coverage results per notebook size in cells (F1 for defs, uses, and producer→consumer edges).

remain high on the multi-kernel demo and RAVL; the main drops occur in R-heavy notebooks and in cases where the gold rule set treats function parameters as definitions or where file paths are constructed via variables rather than literals.

- **exhaustive_python:** defs/uses/edges all F1 = 1.0000.
- **multi_kernel_demo:** defs F1 = 1.0000; uses F1 = 0.9697; edges F1 = 1.0000. One missing use originates from an R cell (`read.csv`).
- **exhaustive_r:** defs F1 = 0.5000; uses F1 = 0.6250; edges F1 = 0.5000. Losses reflect limitations in the current R parser (e.g., `df$temperature` style uses and missed downstream dataflow edges).
- **SecretsProvider_demo:** defs F1 = 1.0000; uses F1 = 0.6000; edges F1 = 1.0000. Errors include false-positive uses from annotated types and missed uses for selected identifiers.
- **migrate_secrets:** defs F1 = 0.9167; uses F1 = 0.7273; edges F1 = 0.0000. Edge failures are explained by file paths being stored in variables (non-literal `open(pathVar)`).
- **RAVL:** defs F1 = 0.9782; uses F1 = 0.9598; edges F1 = 0.9959. Recall losses are dominated by function parameters treated as defs in gold and variable-driven file I/O.
- **RAVL_R_source:** defs F1 = 0.4892, other outputs empty because the kernel name `conda-env-ravl-r` is not currently detected as R by the capture layer; gold exposes this as a kernel-detection gap.

5.5.2 O2: Comprehension & efficiency

We collected time-to-answer for $n = 10$ participants under both conditions (C0 and C1), with counter-balanced condition order and task-set swapping (A/B). Total time per condition is computed as the sum of the three tasks performed in that condition.

Round 1 (prototype). In Round 1, median total time was 144.5 s in the baseline (C0) versus 111.5 s with CellScope (C1), corresponding to a median reduction of 18.4%. A paired Wilcoxon signed-rank test on per-participant total time indicates a statistically significant improvement ($p = 0.0488$; two-sided), with a medium-to-large effect ($r \approx 0.63$). Notably, not all participants were faster with the prototype, consistent with qualitative feedback about difficulty finding filter results and friction in analysing multiple notebooks.

Round	C0 median [IQR] (s)	C1 median [IQR] (s)	Median Δ (%)	Wilcoxon p; r
R1 Prototype	144.5 [116, 167]	111.5 [95, 147]	18.4%	0.0488; 0.63
R2 Post-improvement	144.5 [116, 167]	53.0 [44, 62]	67.7%	0.00195; 0.89

Table 5.3: O2 total time-to-answer (sum of three tasks) for baseline (C0) vs. CellScope (C1), before and after post-evaluation improvements.

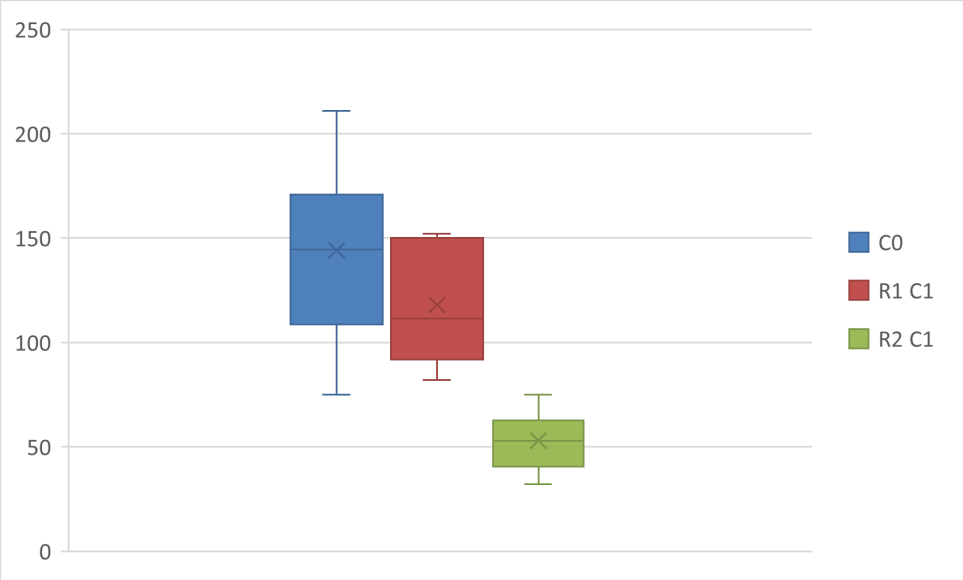


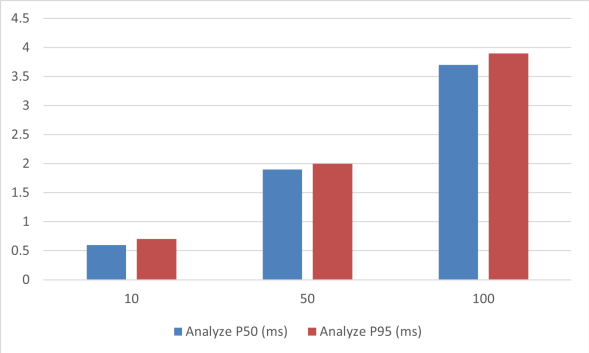
Figure 5.2: O2 total time-to-answer (sum of three tasks) for baseline notebook navigation (C0), CellScope Round 1 (R1 C1), and CellScope Round 2 after improvements (R2 C1).

Round 2 (post-improvement). After implementing the improvements in Table 5.2, CellScope times decreased substantially. Median total time remained 144.5 s in the baseline (C0) and dropped to 53.0 s with CellScope (C1), corresponding to a median reduction of 67.7%. The paired Wilcoxon signed-rank test indicates a significant improvement ($p = 0.00195$; two-sided), with a large effect ($r \approx 0.89$). In Round 2, all participants were faster with CellScope than with baseline.

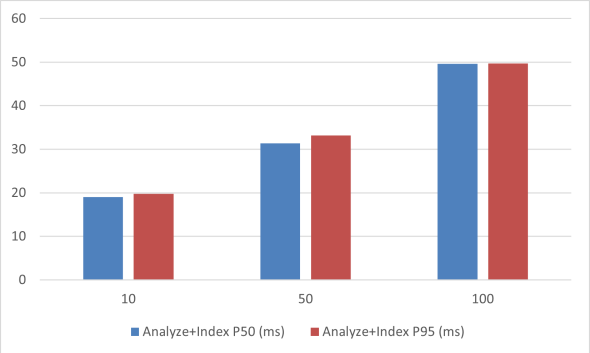
Improvement magnitude (Round 1 → Round 2). Comparing CellScope totals pre- and post-improvement, the median CellScope time decreased from 111.5 s to 53.0 s (median relative improvement 55.0%). This supports the claim that the post-evaluation changes materially improved usability and navigation efficiency.

Distribution of participant times. Figure 5.2 shows the distribution of total time-to-answer (sum of three tasks) across participants. The baseline (C0) has the widest spread, reflecting variation in notebook navigation strategies. In Round 1, CellScope reduces median time but retains variance consistent with observed friction points in finding filter results and managing multi-notebook analysis. After the post-evaluation improvements, Round 2 times are both lower and more tightly clustered, indicating that the changes improved not only central tendency but also consistency across participants. We report a single baseline distribution because the baseline interface and tasks remained unchanged across rounds.

Observed export task (not timed). In addition to the timed navigation tasks, we included an observed task to validate the usability of the RO-Crate export workflow. Participants were instructed to export an RO-Crate containing a specified notebook, without being told where the output would be written. This task was not included in the time-to-answer analysis because it differs in structure from the dependency/comprehension prompts and because timing was not recorded consistently. Nonetheless, across sessions participants successfully completed the export and located the resulting RO-Crate without assistance, suggesting that the export interaction is discoverable and does not introduce practical friction in the end-to-end workflow.



(a) Analyse latency (P50/P95).



(b) Analyse+Index latency (P50/P95).

Figure 5.3: O3 latency versus notebook size (10/50/100 cells). Indexing adds overhead but remains within interactive bounds at the tested scale.

Order effects. When CellScope was used first (C1→C0), baseline times were lower than in the C0→C1 group, indicating learning; however CellScope remained consistently faster after the post-improvement changes.

5.5.3 O3: Real-time & scale results

Local analysis and export. Figure 5.3 reports local latency as notebook size increases from 10 to 100 cells. Analyse latency remains low across sizes, supporting interactive use even on larger notebooks. Adding indexing increases latency as expected, but the P95 values remain within the thresholds defined in the evaluation plan, indicating that optional indexing can be enabled without breaking interactivity at the tested scale.

All notebooks measured are well below the interactive thresholds proposed in the evaluation plan. P95 timings include:

- **RAVL:** analyse ≈ 0.092 s; analyse+index ≈ 0.100 s.
- **multi_kernel_demo:** analyse ≈ 0.0033 s; analyse+index ≈ 0.0295 s.
- **exhaustive_python:** analyse ≈ 0.0022 s; analyse+index ≈ 0.0261 s.
- **exhaustive_r:** analyse ≈ 0.0014 s; analyse+index ≈ 0.0221 s.

These local timings establish that capture and indexing can be performed interactively; we next evaluate the query-time performance of the indexed store.

SPARQL latency on a loaded store. After indexing all example notebooks, the store contained 2166 triples. Median query latency (P50) ranged from 14 ms to 41 ms across templates, and P95 remained below 210 ms, including the worst-case `list_graphs` template. These results indicate that indexing supports interactive discovery queries at the tested scale.

5.5.4 Metadata adequacy for applications

We assess metadata richness pragmatically by demonstrating that the captured and confirmed fields are sufficient to support the thesis' target applications: (i) answering provenance/dependency questions without execution (O2 tasks), (ii) packaging an inspectable and portable research object (RO-Crate export), and (iii) enabling discovery queries over indexed crates (O3 SPARQL templates). Rather than optimising for maximal metadata, CellScope targets "sufficient metadata" for these workflows: structure (cell relations), context (artefacts and snapshots), and optional semantic enrichment via confirm-first and project-specific flavours.

5.6 Threats to validity

Small samples and domain specificity limit generalisation; AST under-approximates dynamic flows; tool familiarity varies. We did not analyse task correctness rates in this iteration; results therefore capture efficiency rather than accuracy. Because tasks were split into two sets (A/B), residual differences in task

difficulty may influence timing despite counterbalancing; we mitigate this by using matched task sets and equal assignment of sets to each condition through order counterbalancing. We mitigate by (i) using a within-subjects design and counterbalancing; (ii) reporting per-notebook and per-task results (avoiding over-aggregation); and (iii) preregistering acceptance thresholds (H1-H3) [3, 6].

Two-round iteration effects (O2). Round 2 re-validates CellScope after targeted usability improvements. Because the baseline interface did not change, and repeating baseline would add participant fatigue, Round 2 primarily tests the improved tool efficiency under the same task structure. Residual learning and memory effects may remain despite counterbalancing and task-set swapping; therefore, Round 2 results should be interpreted as evidence of improvement under comparable conditions rather than a fully independent re-sampling.

5.7 Materials and availability

We provide a single public repository containing both the CellScope source code and the full set of evaluation materials required to reproduce the reported results [24]. The repository includes: (i) the sanitized notebooks used for coverage checks and the user study, (ii) gold templates and predicted capture outputs for O1, (iii) timing logs/spreadsheets and study protocol materials for O2, (iv) benchmark scripts and raw results for O3, and (v) representative exported RO-Crates with the offline HTML graph viewer. A reproduction guide (`REPRODUCE.md`) documents the commands to rerun each experiment and regenerate the tables/figures reported in this thesis.

Chapter 6

Discussion

This chapter reflects on what was achieved, how the results relate to the research questions and use cases introduced in Chapter 3, and what we learned while building CellScope.

Key findings from evaluation. Overall, evaluation provides three main findings. First, CellScope's static capture is accurate for Python notebooks and partially accurate for polyglot settings, with failures concentrated in R parsing and in variable-driven file I/O that cannot be resolved statically (O1). Second, the integrated views substantially improve navigation efficiency after targeted usability fixes: median time-to-answer dropped from 144.5 s (baseline) to 53.0 s with the improved CellScope, a 67.7% reduction (O2). Third, analysis, export, and indexing remain within interactive bounds at the tested scales, and SPARQL query latency stays below the pre-defined thresholds (O3).

6.1 Achievement of objectives

We evaluate the project against the requirements (R1–R4) and the evaluation objectives (O1–O3) introduced in Chapter 5. The discussion below grounds each requirement in the empirical results: coverage scores and failure modes (O1), user navigation efficiency before/after iterative improvements (O2), and interactive performance for analysis and optional indexing (O3).

R1 (Capture). CellScope captures a draft of intra-notebook structure at the cell level (defs/uses, file reads/writes) and derives producer-to-consumer edges. Evaluation (O1) shows that this static-first approach is highly accurate on controlled Python notebooks (perfect F1 on `exhaustive_python`) and remains strong on mixed settings such as `multi_kernel_demo` and `RAVL`, where most relations are recovered but recall drops appear in specific patterns. The dominant failure modes reveal the boundary of static capture: (i) differences between gold rules and implementation for parameter handling, (ii) variable-driven file paths (non-literal I/O) that cannot be resolved without execution or stronger value inference, and (iii) gaps in R support, including kernel identification issues and missed R-specific access patterns (e.g., `df$col`). These results support static-first capture as a practical baseline while motivating targeted extensions for polyglot and dynamic behaviour.

R2 (Package). CellScope exports a portable RO-Crate that bundles the notebook, per-cell snapshots, materialised artifacts, graph exports, and JSON-LD metadata. Evaluation supports this requirement in two ways. First, in the user study we included an observed (non-timed) export task: participants successfully exported an RO-Crate for a specified notebook and located the resulting package without assistance, indicating the workflow is discoverable in practice. Second, performance results (O3) show that analyse, export, and analyse+index remain within interactive bounds at the tested scales, and that indexing enables fast discovery queries: median SPARQL latency stays in the tens of milliseconds and P95 remains below the defined threshold on the loaded store. Together, these results suggest that packaging is not only standards-based but also practical to execute during normal notebook work.

R3 (Explore). The prototype provides complementary list/filter exploration in JupyterLab and an offline graph view in the exported package, aligning with the goal of supporting both "structure at a glance" and precise discovery. This requirement is directly supported by the comprehension/efficiency

study (O2). In Round 1, CellScope yielded a modest but significant reduction in time-to-answer versus baseline, with qualitative feedback pointing to findability issues (e.g., difficulty locating filter matches) and friction when analysing multiple notebooks. After implementing targeted fixes (highlighting and pinning filter matches, reducing multi-notebook workflow friction, and improving session behaviour), Round 2 produced a large improvement: median total time dropped from 144.5 s (C0) to 53.0 s (C1), a 67.7% reduction, and results became more consistent across participants. This indicates that the view concept is effective, but that small interaction details in search and multi-notebook navigation materially determine whether the benefit is realised.

R4 (Scope and polyglot readiness). The implementation is Python-first but supports multi-kernel notebooks by representing cells with their kernel/language and by inferring cross-kernel dependencies through materialised file hand-offs. O1 results show this strategy works when exchanges are explicit: the multi-kernel demo maintains high coverage for inferred producer-to-consumer edges, illustrating that file materialisation is a robust cross-language bridge without runtime instrumentation. However, O1 also exposes polyglot gaps that must be addressed for broader readiness: R kernel detection can fail for non-standard kernel names, and static parsing for R misses common patterns and dynamic behaviour. In practice, these results suggest that polyglot support is strongest when workflows externalise hand-offs as files, and weakest when dependencies remain implicit in kernel state or dynamically constructed paths.

6.2 Use case reflection

We revisit the use cases from §3.3 to discuss which user goals are met and where gaps remain.

UC1 (VRE integrator). The core pipeline is independent of VRE-specific metadata, while personalisation hooks enable "flavors" that specify required object types and fields. This supports platform governance without forcing platform coupling into the capture logic.

UC2 (Virtual lab developer). The cell-level dependency summary and graph/list views support the exact navigation goals reflected in the O2 tasks: locating where a variable is defined, tracing downstream impact, and identifying producers/consumers of artifacts across notebooks. The user study shows that, after usability fixes, participants answered these provenance and dependency prompts substantially faster with CellScope than with baseline navigation, indicating that the views are effective for impact analysis and "where did this come from?" questions without executing code. Remaining gaps align with O1 limitations: when dependencies are expressed through dynamically built paths or R-specific semantics, the captured structure can under-approximate the true relationships.

UC3 (External collaborator). The RO-Crate export supports inspection outside the originating VRE: collaborators can open the offline HTML graph locally, inspect JSON-LD metadata, and review per-cell snapshots and packaged artifacts. The observed export task in O2 indicates that exporting and locating the resulting crate is straightforward for users, which strengthens the practical viability of this use case. Full rerunnability remains contingent on access to inputs (local artifacts or resolvable external references) and a compatible execution environment; nonetheless, the crate provides a concrete, portable "explanation bundle" that reduces the effort needed to understand and safely reuse parts of a notebook.

6.3 Lessons learned

Static-first capture is a practical baseline. For notebook authoring, static analysis provides a strong cost-benefit trade-off: it is fast, safe, and explainable, and it already answers many provenance questions that users have (e.g., "where did this value come from?"). The main limitation is under-approximating dynamic behaviour; the design therefore benefits from making uncertainty visible (e.g., by relying on file materialisation for cross-kernel links).

Portability improves when views are generated from stored artifacts. Embedding an offline graph in the exported package significantly reduces friction for external collaborators: they can inspect dependencies without installing platform services. This reinforces the generate-then-visualise principle from Chapter 3.

Lightweight author hints add disproportionate value. A small review step (roles and file descriptors) improves interpretability and reuse without turning the workflow into a heavy metadata authoring task. This fits the confirm-first lifecycle while keeping the default path usable.

Manual metadata entry remains a bottleneck. While CellScope can extract substantial structural context automatically (defs/uses, producer→consumer relations, and file reads/writes), high-quality descriptive metadata still depends on manual user input. This is especially true for domain-specific fields, which vary significantly between projects and communities and therefore cannot be hard-coded into a single universal schema. CellScope mitigates this through a lightweight confirm-first step and integrator-defined personalisation, but it does not eliminate the underlying effort: users still need to supply or verify the semantic context that makes artifacts reusable and results reproducible. This directly motivates future work on persisting confirm-first values across analyses and on optional LLM-assisted drafting of metadata and provenance hints, where suggestions remain user-confirmed and are recorded explicitly in the exported RO-Crate (Chapter 8).

Indexing should remain a projection, not the source of truth. Treating the triple store as an index (derived from the RO-Crate) preserves author portability and reduces lock-in. It also clarifies responsibility: the crate is the canonical package; the index is an optimisation for discovery.

Iterative evaluation exposed where usability leverage actually is. The two-round O2 study showed that the conceptual design (graph + list/filter) was not sufficient on its own: in Round 1, benefits were limited by small interaction frictions. After implementing narrow, concrete fixes—making filter matches visually salient, pinning exact matches, smoothing multi-notebook analysis, and improving session behaviour—the measured efficiency gains became large and consistent. This suggests that, for notebook navigation tools, the "last mile" of findability and workflow friction can dominate outcomes more than the underlying analysis algorithms once a reasonable baseline capture is in place.

6.4 Implications

The results suggest that cell-level, storage-backed metadata can materially improve notebook work in two complementary ways: (i) it enables fast, execution-agnostic navigation of definitions, dependencies, and artifact provenance (large time-to-answer reductions after usability refinements), and (ii) it produces a portable package that preserves context for collaborators without requiring platform services. At the same time, O1 pinpoints the main technical barriers to broader applicability: static capture under-approximates dynamic patterns such as variable-driven file paths, and polyglot fidelity depends on reliable kernel detection and language-specific parsing. These implications motivate future extensions focused on richer path/value resolution, improved R support, and stronger cross-kernel provenance beyond file materialisation, while preserving the core "generate then visualise" and "crate as source of truth" principles.

Chapter 7

Conclusions

This thesis introduced CellScope, a metadata-driven workflow for capturing, packaging, and exploring notebook-internal structure at the cell level. CellScope aims to improve understanding, reuse, and collaboration around Jupyter notebooks by (i) extracting definitions/uses and producer→consumer relations without executing code, (ii) exporting a portable RO-Crate that preserves artifacts and metadata, and (iii) providing complementary exploration views in JupyterLab and offline.

7.1 Answers to research questions

This section answers the research questions from §1.3, referring to evaluation results in Chapter 5 where needed.

Main RQ: How can richer cell-level metadata and contextual information about digital objects in Jupyter notebooks improve the understandability, reproducibility, and reuse of computational results? This thesis shows that capturing and externalising cell-level structure and object context into portable artifacts measurably improves understandability and supports reuse-oriented workflows. CellScope makes dependencies and provenance queries (e.g., where values are defined, which cells depend on them, and where files are produced/consumed) accessible without executing the notebook, and a user study shows substantially faster completion of these navigation tasks with the improved interface. Reproducibility and reuse are supported by packaging: the exported RO-Crate preserves notebooks, materialised outputs, and machine-readable metadata for offline inspection and indexing. While CellScope does not guarantee full rerunnability of arbitrary notebooks, it provides an "explanation bundle" that reduces the effort needed to reconstruct context and identify the inputs and steps that produced results.

RQ1: Which kinds of cell-level metadata and contextual information about digital objects do notebook authors and reusers need in order to understand and reliably reproduce results? The evaluation tasks and observed user behaviour suggest that a small set of metadata types delivers most practical value: (i) symbol definitions and uses at cell granularity, (ii) explicit producer→consumer relations for variables and materialised artifacts, (iii) file reads/writes with paths and basic descriptors, and (iv) lightweight per-cell context (kernel/language, cell type, and captured snapshots of intermediate outputs). These enable common author/reuser questions such as impact analysis (what breaks if this changes), reverse provenance (what produced this artifact), and cross-notebook handoffs. Domain-specific metadata is often essential for reuse because it provides the semantic context that generic capture cannot infer (e.g., units, spatial/temporal extent, coordinate reference systems, instrument identifiers, quality flags, and domain parameter conventions). However, the required fields and vocabularies vary substantially across projects and communities, and NaaVRE does not impose a single domain schema. For this reason, CellScope does not hard-code domain-specific fields as part of the core metadata model. Instead, it supports *personalisation* through lightweight author hints and guidance that allow projects to declare the object types and metadata fields they consider mandatory. This keeps the default workflow portable and broadly applicable while making it straightforward to add domain-specific metadata when a particular VRE or project requires it. The main gap is dynamic behaviour: when file paths are constructed via variables or dependencies remain implicit in runtime state, static capture under-approximates what users may need for fully reliable reproduction.

RQ2: How can this metadata be structured and packaged into a portable, web-native representation that tools can index and humans can inspect? CellScope demonstrates a practical packaging strategy: represent captured metadata as web-native, linked-data-friendly artifacts and bundle them with the notebook and materialised outputs in an RO-Crate. Evaluation shows that this approach is operationally feasible (export is usable in practice) and performant enough for interactive use; indexing remains a projection derived from the crate rather than the source of truth. This structure supports both human inspection (offline HTML graph and packaged JSON-LD) and tool-based discovery (SPARQL queries over an indexed store), while keeping the author-controlled crate as the canonical unit of sharing.

RQ3: What interface within JupyterLab best helps users see and navigate this metadata about digital objects directly from the notebook? The results indicate that no single view is sufficient: the most effective interface combines (i) a list/filter view for targeted lookup and lightweight metadata search, and (ii) an interactive dependency graph for "structure at a glance" and provenance traversal. In the user study, CellScope reduced time-to-answer compared to baseline navigation, and the effect became large and consistent after improvements that increased findability (highlighting/pinning filter matches) and reduced workflow friction (multi-notebook analysis and session behaviour). This suggests that the winning interface pattern is a hybrid of search-driven and graph-driven navigation, with strong emphasis on small interaction details that make metadata discoverable at the moment of need.

7.2 Closing remarks

CellScope demonstrates that lightweight, storage-backed metadata at cell granularity can bridge notebook authoring and reuse: users gain faster dependency-oriented navigation, and collaborators gain a portable package for inspection outside the originating environment. The primary remaining challenges are improving fidelity for dynamic patterns (e.g., variable-driven file paths) and strengthening polyglot support beyond file materialisation, which motivates the future work outlined in the following chapter. Overall, the contribution is a workflow that connects understandability (navigation), reproducibility support (captured context and artifacts), and reuse (portable packaging and optional indexing) through the same cell-level metadata layer.

Chapter 8

Future work

This chapter outlines extensions motivated by the lessons learned in Chapter 6 and the limitations observed in evaluation (Chapter 5). The overarching goal is to improve fidelity for polyglot and dynamic notebook patterns while preserving the static-first, portable, "crate-as-source-of-truth" design.

8.1 Capture fidelity and polyglot support

Strengthen the R parser beyond regex-based heuristics. O1 exposed that R notebooks are a primary source of recall loss, including common access patterns (e.g., `df$col`) and edges that are obvious to humans but difficult to recover with lightweight rules. A next step is to replace the current heuristic layer with an AST-based R front-end (e.g., leveraging `parse()` output and normalised traversal) to extract definitions, uses, and calls more robustly. The goal is not perfect equivalence with dynamic evaluation, but to reduce systematic misses on idiomatic R syntax and to better align with the gold rule set for R notebooks.

Improve kernel identification and language mapping. Evaluation showed that polyglot capture can fail when kernel names do not match expected patterns, resulting in entire notebooks producing empty outputs even when code is present. Future work should implement a more reliable kernel-to-language mapping, combining (i) inspection of notebook metadata (kernelspec and language info), (ii) a configurable alias table (e.g., mapping environment-specific kernel names to languages), and (iii) fallback detection based on cell magics or language markers. This preserves portability while preventing silent failure modes that currently reduce coverage in heterogeneous environments.

Extend cross-kernel provenance beyond file materialisation. CellScope currently exposes cross-kernel relations primarily through materialised file hand-offs, which works well when workflows externalise intermediate results. Future work could introduce additional bridges without requiring full execution tracing, such as recognising structured exchange patterns (e.g., JSON passed through environment variables, parameter files, or common IPC wrappers) and capturing declared dependencies between notebooks in a project. Where execution-time instrumentation is available, a hybrid mode could optionally reconcile static edges with runtime evidence while keeping static capture as the default.

8.2 Confirm-first UX and metadata persistence

Persistent confirm-first metadata inputs. The confirm-first step provides high leverage (Chapter 6) by allowing lightweight author hints without forcing heavy metadata authoring. However, the current implementation requires users to re-enter the same metadata fields every time they re-analyse a notebook, which creates unnecessary friction. Future work should persist confirm-first inputs per notebook (and optionally per project/workspace) so that previously provided values are pre-filled on subsequent analyses. This persistence should be explicit and inspectable: users should be able to review, edit, and reset stored values, and exports should record which metadata values were user-provided versus automatically extracted.

Project-level defaults and "flavors" integration. To support highly variable domain-specific metadata requirements across projects, confirm-first should integrate more directly with the "flavor" mechanism. Future work can allow integrators to define a project template that specifies required fields, validation rules (e.g., required/optional, controlled vocabulary), and sensible defaults. This keeps the core pipeline domain-agnostic while making domain-specific metadata capture fast and consistent for communities that need it.

LLM-assisted metadata and provenance hints (optional). A complementary direction is to use a large language model (LLM) as an *assistive layer* rather than as a source of truth. Given notebook text, cell code, and the artifacts already captured by CellScope, an LLM could (i) propose candidate metadata values for confirm-first fields (e.g., suggested titles/descriptions, likely units or parameter semantics when present in comments), (ii) suggest human-readable summaries of what a cell produces and consumes, (iii) recommend likely producer→consumer links in cases where static analysis is uncertain (e.g., variable-driven paths), and (iv) map project-specific terminology to a chosen schema. Importantly, these suggestions should be presented as drafts that users can accept or edit, and accepted values should be stored persistently to reduce repeated manual entry. This could improve usability and coverage without weakening the thesis' core principles: static capture remains the baseline, confirm-first remains the evaluation gate, and the exported RO-Crate records which metadata was inferred, suggested, or explicitly confirmed.

8.3 Handling dynamic patterns without full execution

Value inference for variable-driven file paths. A dominant O1 failure mode is variable-driven file I/O, where paths are constructed through variables or simple string operations and therefore are not visible as literals. A practical extension is to add lightweight, conservative value inference for common patterns (e.g., constant propagation within a cell, f-strings with constant components, `os.path.join` with known literals, and simple environment-variable expansion). The system should surface uncertainty explicitly, distinguishing inferred paths from confirmed literals, to preserve the explainable nature of static-first capture.

Optional runtime reconciliation as an advanced mode. For users who can execute notebooks (or run trusted pipelines), an optional mode could reconcile static predictions with runtime evidence (e.g., confirmed file paths, actual produced artifacts, and dynamically resolved dependencies). This should remain opt-in to preserve CellScope's safety and portability guarantees, but it would improve fidelity for notebooks where dynamic behaviour is central.

8.4 Indexing and scalability extensions

Richer query templates and higher-level discovery features. O3 indicates that indexing supports interactive discovery at the tested scale. Future work can add higher-level query templates that match real user questions (e.g., "show all artifacts produced by notebook X", "find all notebooks that write a given dataset", "trace provenance of this output across notebooks") and expose them through UI shortcuts rather than raw SPARQL. This builds on the lesson that interaction design and findability strongly determine realised benefits.

Incremental indexing and cache invalidation. As projects grow, re-indexing entire crates on every change becomes inefficient. Future work should support incremental indexing keyed by notebook hash, cell hash, and artifact digest, updating only affected subgraphs. This also supports the "crate as source of truth" principle: the index can be safely rebuilt or repaired from stored crates, while incremental updates improve responsiveness during active development.

8.5 Summary

Future work prioritises extensions that preserve the core principles validated in this thesis: static-first capture for fast feedback, portable RO-Crates as the canonical package, and derived indexing for discovery. The lessons learned in Chapter 6 suggest that usability details (findability, workflow friction, and

persistence of small author inputs) can be as important as analysis algorithms, and the evaluation results highlight where targeted technical improvements (R parsing, kernel detection, and variable-driven path inference) will yield the largest gains in coverage and trust.

Acknowledgements

First, I thank my academic supervisor, Dr. Zhiming Zhao, for selecting this topic and for providing resources and support from his research group. His academic guidance helped shape the direction of the work and strengthened the research approach behind it.

I am also deeply grateful to my daily supervisor, Koen Greuell, for his consistent support throughout the development process. Our weekly discussions were invaluable for refining the tool, making practical design decisions, and iterating on evaluation. His feedback and encouragement made a major difference in keeping the project focused and progressing steadily.

I would also like to thank the teams at both the Multiscale Networked Systems research group and LifeWatch ERIC for their feedback on the initial problem definition and on the first prototype of the tool. Their comments helped clarify early priorities and improved the direction of the implementation from the start.

I would like to thank the teachers in the programme for equipping me with the research skills needed to carry out this work, from framing questions and evaluating systems to communicating results clearly. I also thank the programme coordinators for creating a challenging but enjoyable environment, and for organising social events that made the experience feel welcoming and connected.

Finally, I thank my friends and family for their support and patience throughout the thesis. Their encouragement, and the feedback they offered from many different perspectives, helped me stay motivated and grounded throughout the process.

Bibliography

[1] J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire, "A large-scale study about jupyter notebooks," *Proceedings of the 16th International Conference on Mining Software Repositories (MSR)*, pp. 507–517, 2019.

[2] J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire, "Understanding and improving the quality and reproducibility of Jupyter notebooks," *Empirical Software Engineering*, vol. 26, no. 4, p. 65, 2021. DOI: 10.1007/s10664-021-09961-9.

[3] A. Rule, A. Tabard, and J. D. Hollan, "Exploration and explanation in computational notebooks," in *CHI*, ACM, 2018, 32:1–32:12.

[4] T. Kluyver, B. Ragan-Kelley, F. Pérez, and et al., "Jupyter notebooks – a publishing format for reproducible computational workflows," in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, IOS Press, 2016, pp. 87–90.

[5] T. Lebo *et al.*, "Prov-o: The prov ontology," W3C, W3C Recommendation, Apr. 2013. [Online]. Available: https://www.w3.org/TR/prov-o/.

[6] M. Baker, "1,500 scientists lift the lid on reproducibility," *Nature*, vol. 533, no. 7604, pp. 452–454, 2016.

[7] J. P. A. Ioannidis, "Why most published research findings are false," *PLoS Medicine*, vol. 2, no. 8, e124, 2005.

[8] G. K. Sandve, A. Nekrutenko, J. Taylor, and E. Hovig, "Ten simple rules for reproducible computational research," *PLOS Computational Biology*, vol. 9, no. 10, e1003285, 2013. DOI: 10.1371/journal.pcbi.1003285.

[9] M. D. Wilkinson and et al., "The fair guiding principles for scientific data management and stewardship," *Scientific Data*, vol. 3, p. 160 018, 2016.

[10] Z. Zhao *et al.*, "Cloud-native data analytics workflows for science: From devops to DataOps," *Software: Practice and Experience*, vol. 52, no. 8, pp. 1784–1808, 2022. DOI: 10.1002/spe.3091.

[11] S. Soiland-Reyes *et al.*, "Packaging research artefacts with ro-crate," *Data Science*, vol. 5, no. 2, pp. 97–138, 2022. DOI: 10.3233/DS-210053.

[12] R.-C. Community, *Ro-crate specification 1.1*, https://www.researchobject.org/ro-crate/1.1/, Accessed 2025-09-05, 2025.

[13] J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire, "Noworkflow: A tool for collecting, analyzing, and managing provenance from python scripts," in *PVLDB*, vol. 10, 2017, pp. 1841–1844. DOI: 10.14778/3137765.3137810.

[14] T. McPhillips, S. Bowers, D. Zinn, and B. Ludäscher, "Yesworkflow: A user-oriented, language-independent tool for recovering workflow information from scripts," in *IPAW*, Springer, 2015, pp. 230–244.

[15] A. Hagberg, D. Schult, and P. Swart, "Exploring network structure, dynamics, and function using networkx," in *SciPy*, 2008.

[16] *Vis.js network*, https://visjs.org/, 2024.

[17] M. R. Crusoe, M. Abueg, K. Alam, *et al.*, "Workflowhub: A registry for fair computational workflows," in *2022 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*, IEEE, 2022, pp. 2685–2692. DOI: 10.1109/BIBM55620.2022.9995761.

[18] K. Wolstencroft, S. Owen, A. R. Williams, C. Goble, S. Soiland-Reyes, *et al.*, "Fairdom-seek: A systems biology data and model management platform," *Bioinformatics*, vol. 33, no. 11, pp. 1748–1750, 2017. DOI: 10.1093/bioinformatics/btx265.

[19] J. Boon, "Enhancing r code containerization through automated input/output detection and type inference," Master Software Engineering thesis, M.S. thesis, University of Amsterdam, Aug. 2024.

[20] F. Maali, J. Erickson, and P. Archer, "Data catalog vocabulary (DCAT)," W3C, W3C Recommendation, Jan. 2014. [Online]. Available: https://www.w3.org/TR/vocab-dcat/.

[21] S. J. D. Cox *et al.*, "Data catalog vocabulary (dcat)—version 2," W3C, W3C Recommendation, Feb. 2020. [Online]. Available: https://www.w3.org/TR/vocab-dcat-2/.

[22] S. Leo *et al.*, "Recording provenance of workflow runs with RO-Crate," *PLOS ONE*, vol. 19, no. 9, e0309210, 2024. DOI: 10.1371/journal.pone.0309210.

[23] E. Afgan *et al.*, "The galaxy platform for accessible, reproducible and collaborative biomedical analyses: 2018 update," *Nucleic Acids Research*, vol. 46, no. W1, W537–W544, 2018. DOI: 10.1093/nar/gky379.

[24] G. Camacho Hortelano, *Cellscope: Capturing cell-level metadata in jupyter notebooks*, GitHub repository. Accessed 2025-11-07, 2025. [Online]. Available: https://github.com/PixelVolt38/cellscope.

[25] Y. W. V. Christou and Z. Zhao, "Towards a knowledge graph enhanced automation and collaboration framework for digital twins," M.S. thesis, University of Amsterdam, 2023.

[26] C. Community, *Cf (climate and forecast) metadata conventions*, https://cfconventions.org/, Accessed 2025-09-05, 2025.

[27] C. Community, *Cf standard name table*, https://cfconventions.org/standard-names.html, Accessed 2025-09-05, 2025.

[28] L. Quaranta, D. Di Castro, S. Scardapane, and F. A. Lisi, *Eliciting best practices for collaboration with computational notebooks*, arXiv preprint arXiv:2202.07233, 2022. [Online]. Available: https://arxiv.org/abs/2202.07233.

[29] D. Garijo, C. Goble, S. Soiland-Reyes, S. Owen, M. R. Crusoe, *et al.*, "Workflowhub: A registry for fair computational workflows," *Scientific Data*, 2025, in press.

[30] E. Afgan, J. Taylor, A. Nekrutenko, *et al.*, "The galaxy platform for accessible, reproducible and collaborative biomedical analyses: 2024 update," *Nucleic Acids Research*, vol. 52, no. W1, W83–W90, 2024. DOI: 10.1093/nar/gkae410.

[31] ARDC, *Describo: Ro-crate creation tool*, https://arkisto-platform.github.io/describo/, Accessed 2025-09-05, 2025.

[32] T. McPhillips *et al.*, "Yesworkflow: A user-oriented, language-independent tool for recovering workflow information from scripts," in *International Provenance and Annotation Workshop (IPAW)*, ser. Lecture Notes in Computer Science, vol. 8628, Springer, 2015, pp. 230–244. DOI: 10.1007/978-3-319-16462-5_19.

[33] S. Samuel and B. König-Ries, "Provbook: Provenance-based semantic enrichment of interactive notebooks for reproducibility," in *ISWC 2018 Posters & Demos (CEUR-WS Vol. 2180)*, Cited in this thesis as "Kau2020"; ProvBook line of work relevant to 2020–2022 follow-ups, 2018, 57:1–57:4. [Online]. Available: http://ceur-ws.org/Vol-2180/paper-57.pdf.

[34] S. Samuel, F. Löffler, and B. König-Ries, *Reproducemegit: A tool to support reproducibility of machine learning pipelines*, arXiv:2006.12110, 2020. [Online]. Available: https://arxiv.org/abs/2006.12110.

[35] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[36] J. Brooke, "Sus: A "quick and dirty" usability scale," *Usability Evaluation in Industry*, pp. 189–194, 1996.

[37] S. G. Hart and L. E. Staveland, "Development of nasa-tlx (task load index): Results of empirical and theoretical research," in *Advances in Psychology*, vol. 52, Elsevier, 1988, pp. 139–183.